A Project Report

on

PRE-OWNED MERCHANDISE

Submitted to the

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY HYDERABAD

In partial fulfillment of the requirement for the awards of the degree of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING

By

B. HARSHAL	**22TP1A0516**
G. RAJEEV REDDY	**22TP1A0539**
G. VYSHNAVI	**22TP1A0545**
J. MALLIKHARJUN	**22TP1A0561**

UNDER THE ESTEEMED GUIDANCE OF

Mr. G. UDAY KUMAR

M. Tech

Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

SIDDHARTHA INSTITUTE OF ENGINEERING AND TECHNOLOGY

(Approved by AICTE, Accredited by NBA & NAAC A+, Affiliated to JNTU, Hyderabad)
Vinobanagar (V), Ibrahimpatnam (M), R.R. Dist. 501506.

2025-2026

SIDDHARTHA INSTITUTE OF ENGINEERING AND TECHNOLOGY

(Approved by AICTE, Accredited by NBA & NAAC A+, Affiliated to JNTUH)

Vinobanagar, Ibrahimpatnam, Hyderabad, Telangana 501506

2025-2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the project work entitled **"PRE-OWNED MERCHANDISE"** is being submitted by **B. HARSHAL (22TP1A0516), G. RAJEEV REDDY (22TP1A0539), G. VYSHNAVI (22TP1A0545), J. MALLIKHARJUN (22TP1A0561)** in fulfillment for the requirement of the award of **BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING** of the Jawaharlal Nehru Technological University, Hyderabad, during the academic year 2025-2026 is a bonafide record of work carried out under our guidance and supervision.

Signature	Signature	Signature
PROJECT GUIDE	PROJECT COORDINATOR	HEAD OF THE DEPARTMENT
Mr. G. UDAY KUMAR	**Mr. CHANDRA SRINIVAS P**	**Dr. P. SUNITHA**
MTech	**Assistant Professor**	**MTech, Ph.D.**
Assistant Professor	**Department of CSE**	**Professor & HOD**
Department of CSE		**Department of CSE**

INTERNAL EXAMINER **EXTERNAL EXAMINER**

DECLARATION

This is to certify that the work reported in titled **"PRE-OWNED MERCHANDISE"** submitted to the Department of Computer Science and Engineering, SIET**,** Ibrahimpatnam, Ranga Reddy in fulfillment of degree for the award of Bachelor of Technology, is a Bonafide work done by us.

No part of this major project is copied from any books, journals and wherever the portion is taken, the same has duly need referred in the text. The reported results are based on the major project work entirely done by us and not copied from any other source.

We further state to the best of our knowledge that the matter embedded in this project has not been submitted by us in full or partial there of the award of any degree to any other institution or University.

B. HARSHAL	**22TP1A0516**
G. RAJEEV REDDY	**22TP1A0539**
G. VYSHNAVI	**22TP1A0545**
J. MALLIKHARJUN	**22TP1A0561**

ACKNOWLEDGEMENT

First and foremost, our sincere thanks to our Internal Guide **Mr. G. UDAY KUMAR** Assistant Professor, who has given his full efforts in guiding the team, achieving the goals as well as his encouragement to maintain our progress in track.

We indebted to our Major Project Coordinator, **Mr. CHANDRA SRINIVAS P** Assistant Professor, for his excellent guidance, constant inspiration and encouragement in completion of the major project work.

Our Sincere thanks to **Dr. P. SUNITHA,** Head of the Department of CSE for her valuable suggestions and advice during our completion of the major project.

A Sincere thanks to **Dr. P. SEKHAR BABU,** Principal, **SIDDHARTHA INSTITUTE OF ENGINEERING AND TECHNOLOGY, Vinobanagar, Ibrahimpatnam, Hyderabad** for providing this opportunity to carry out the major project work and for his encouragement during the work.

We would like to express our thanks to all the faculty members for having faith in us in every possible way whenever the need arose. On a personal note, we thank our **beloved parents and friends** for their moral support during our major project.

B. HARSHAL	**22TP1A0516**
G. RAJEEV REDDY	**22TP1A0539**
G. VYSHNAVI	**22TP1A0545**
J. MALLIKHARJUN	**22TP1A0561**

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	Abstract	i
	List of figures	ii
1. INTRODUCTION		1-3
	1.1 Motivation	1
	1.2 Problem Definition	2
	1.3 Objective of Project	3
	1.4 Limitations of Project	3
2. LITERATURE SURVEY		4- 6
3. SYSTEM ANALYSIS		7-12
	3.1 Existing Systems	7
	3.1.1 Disadvantages of Existing Systems	7
	3.2 Proposed Systems	8
	3.2.1 Advantages of Proposed System	8
	3.3 Feasibility study of the Proposed System	9
	3.4 Software Requirement Specification	10
	3.4.1 Software Requirements	11
	3.4.2 Hardware Requirements	11

4. SYSTEM DESIGN 12-24

4.1 Input Design 12

4.2 Output Design 13

4.3 Algorithms 13

4.4 Flow Chart 14

4.5 UML Diagrams 15

4.5.1 Class Diagram 16

4.5.2 Use Case Diagram 17

4.5.3 Sequence Diagram 18

4.5.4 Collaboration Diagram 19

4.5.5 Component Diagram 20

4.5.6 Deployment Diagram 21

4.5.7 Activity Diagram 22

4.5.8 Data Flow Diagram 23

5. IMPLEMENTATION

5.1 Method of Implementation 25-30

5.2 Software Specification 31-35

5.2.1 Python 31

5.2.2 Features of Python 33

5.2.3 Libraries in Python 34-35

5.3 Sample Code 36-44

6. SOFTWARE TESTING 45-47

6.1 Implementation and Testing 45

6.2 Types of Testing 46-47

7. RESULTS AND OUTPUT SCREENS 48-55

7.1 Output Screens 48-55

8. APPLICATIONS AND FUTURE ENHANCEMENTS 56-57

8.1 Applications 56

8.2 Future Enhancements 57

9. CONCLUSION 58

REFERENCES 59-60

ABSTRACT

Pre-Owned Merchandise is an innovative approach that enables users to buy or sell products from the convenience of their home via the internet. The traditional reselling market often involves a middleman who purchases items from sellers and resells them, which tends to inflate the final price. Moreover, in many scenarios, customers are required to visit physical stores to complete transactions, leading to time consumption, increased labor, and higher marketing costs. Some stores may facilitate orders over the phone, but there's no assurance that the delivered product will meet customer expectations.

This project aims to address these issues by building a tailored e-commerce platform that allows users to efficiently buy or sell items such as books, phones, and gadgets. Developed using Amazon Web Services (Cognito, API Gateway, S3 Bucket, Lambdas, DynamoDB, Node.js, Serverless Framework, and Express) alongside ReactJS for the frontend, this system promises a seamless user experience.

A case study focused on the resale of books highlights the benefits, as many find it challenging to afford new books at high market prices. Offering second-hand options at reduced costs makes books more accessible to those in need. A comparative study between the existing methods and the proposed system has been conducted using predefined metrics, demonstrating that this cloud-based Pre-Owned Merchandise platform offers a cost-effective and user-friendly solution to the limitations of current systems.

LIST OF FIGURES

Fig No	Figure name	Page no
4.5.1	Class Diagram	17
4.5.2	Use Case Diagram	18
4.5.3	Sequence Diagram	19
4.5.4	Collaboration Diagram	20
4.5.5	Component Diagram	21
4.5.6	Deployment Diagram	22
4.5.7	Activity Diagram	23
4.5.8	Data Flow Diagram	24
4.5.9	System Architecture	25

CHAPTER - 1

INTRODUCTION

In recent years, with the application and popularization of internet technologies, people can make online shopping, online booking and online payment anytime, anywhere, which greatly facilitates people's lives, saves time, and accelerates the pace of the times. Due to the convenience of online shopping, a group of young people like students may purchase a lot of things that they do not particularly need, which results in the idleness of items. Coupled with the lack of economic ability of students, buying a lot of new items cannot afford. Based on this type of demand, a second-hand market has been created that can sell and purchase various idle items. The Second-hand Commodity Trading Platform of this project is a trading website designed for users and based on the C2C model. C2C is Customer to Customer, which means transactions between individuals and individuals. Taobao is using this model, each user can freely publish and purchase goods, has been achieved unimaginable success. The system is also designed and implemented using this model. Through this system, users can register and log in, quickly publish their own information on idle items, and also browse and purchase a variety of products by category, providing users with a fast and convenient online trading platform. The system is mainly used for teacher and student second-hand goods transactions, so that idle items can play their due value, and here you can get the satisfaction and cheap second-hand items you need. This has not only achieved social and economic health and environmental protection, but also promoted social progress.

1.1 Motivation

In today's digital era, the rapid advancement of internet technologies and e-commerce platforms has significantly transformed the way people buy and sell products. Online shopping has become highly convenient, allowing users to purchase items anytime and from anywhere. However, this convenience has also led to a growing trend of over-purchasing, especially among students and young individuals. Many users often buy items that are either rarely used or become unnecessary over time, leading to a large accumulation of idle goods.

Overall, the project is motivated by economic, social, and environmental factors. It aims to create a sustainable ecosystem where resources are utilized efficiently, users benefit financially, and society moves towards an eco-friendlier approach to consumption.

1.2 Problem Definition

Despite the availability of various methods for buying and selling second-hand goods, the current systems in place are largely inefficient, unstructured, and unreliable. Traditional approaches such as local markets, newspaper advertisements, and word-of-mouth communication are limited in reach and accessibility. These methods often require physical presence, making them time-consuming and inconvenient for users.

In recent years, social media platforms have emerged as an alternative for trading used goods. However, these platforms are not specifically designed for e-commerce purposes. As a result, they lack essential features such as proper product categorization, advanced search filters, secure user authentication, and transaction management systems. This leads to disorganized listings, difficulty in finding relevant products, and a lack of trust between buyers and sellers. Another major issue is the absence of a centralized system to manage user data and product information. Without a structured database, it becomes difficult to track product availability, maintain records, or ensure data consistency. Buyers often struggle to verify the authenticity and quality of products, while sellers face challenges in reaching the right audience.

Therefore, the core problem addressed by this project is the lack of a dedicated, secure, and efficient platform for second-hand goods trading. The project aims to solve this problem by developing a web-based application that centralizes all operations, improves user experience, ensures better data management, and enhances trust through structured interactions and feedback systems.

1.3 Objective of Project

1. To develop a centralized web-based platform for buying and selling pre-owned merchandise efficiently.
2. To implement secure user authentication through user registration and login functionalities for buyers and sellers.
3. To enable easy product management, allowing sellers to add product details with images and store them in a structured database.
4. To provide advanced search functionality for buyers to filter products based on category, price range, and location.
5. To incorporate a feedback and rating system that improves transparency, trust, and user experience within the platform.
6. To develop a user-friendly and responsive interface that ensures smooth navigation and enhances the overall user experience.
7. To ensure data integrity and system reliability by minimizing errors and maintaining accurate records of users and products.

1.4 Limitations of Project

1. The platform does not include an integrated online payment system, so transactions must be completed offline.
2. The system is designed mainly for small-scale usage (students/individual users) and may not support large commercial operations.
3. Scalability is limited, making it difficult to handle a very high number of users or product listings efficiently.
4. Lack of secure payment handling may lead to trust issues between buyers and sellers.
5. No implementation of advanced security mechanisms such as encryption, OTP verification, or multi-factor authentication.
6. The platform cannot fully verify the authenticity of users or products.

CHAPTER - 2

LITERATURE SURVEY

Analysis on the development trend of e-commerce

Zhou Xing 2014

Based on the current development status of e-commerce live broadcasting in China, this paper predicts the development prospects of e-commerce live broadcasting on the basis of analysing the successful elements, advantages and disadvantages of e-commerce live broadcasting. This paper points out that e-commerce live broadcasting will become the "standard configuration" of e-commerce, brand and business in the future, which can continue to develop continuously.

Design and implementation of online shopping system based on Java

Yang Xiaojie. 2015

B/S structure (Browser/Server) is one hidden client mode after WEB development. This kind of network structure mode unifies WEB browser as the client-side in order to integrate the core part of system function realization to the server. B/S model simplifies system development, maintenance, and usage. The client only needs one Browser under the B/S model, and the browser interact data with database through Web Server. Since the B/S model has such huge advantages, this online shopping system is based on this model.

The system through the MVC (Model, View, and Controller) framework integrate network of online shopping system, completing the control layer management, processing data access. Finally, the system can meet online shopping requirements basically, and it’s also a good application of e-commerce. Customers can know the relevant information of products at any time, and they can have mass commodity information. Besides, the drawbacks of traditional shopping such as geographical and time limitations have been solved completely.

Java information system management and development

Zhang Like. 2014

Everyday an out-sized number of individuals visits any city in a country for their higher studies, business development, job opportunities, tourism, etc. For those newly arriving people, system of the city was unknown. For that there is a need of a city guide to know about that city and to visit places in that city. Now-a-days, people prefer searching in internet rather than enquiring the locals of the city or hiring a city guide.For such inconvenience we developed a web based product used to store the details of the city and helps all the users who visits our website. The website contains the entire information about particular city like places to be visited,site maps, route maps, business environment, job portal, information about organization that provide transport, hospitality and total history of the city.

Wide deep learning for recommender systems

H. T. Cheng, 2016

Recommendation systems based on Deep Learning have recently led to significant progress in different application domains. Most models are influenced by hyperparameter optimization or tuning, the stability of training, and architecture configuration. Hence, in the present study, we introduced a Deep Learning model, which is named RHMM, for recommender systems, by using the Hidden Markov Model and Artificial Neural Networks. The model selection technique is applied to optimize the bias-variance tradeoff of the expected prediction. The model aggregation technique is used to improve the robustness and accuracy of training. Experiment results showed that our Deep Leaning model led to significant improvement over benchmarking models.

Predicting the price of used cars using machine learning techniques

S. Pudaruth 2014

In this paper, we investigate the application of supervised machine learning techniques to predict the price of used cars in Mauritius. The predictions are based on historical data collected from daily newspapers. Different techniques like multiple linear regression analysis, k-nearest neighbours, naïve bayes and decision trees have been used to make the predictions. The predictions are then evaluated and compared in order to find those which provide the best performances. A seemingly easy problem turned out to be indeed very difficult to resolve with high accuracy.

Real-time automatic price prediction for ebay online trading

I. Raykhel, and D. Ventura, 2019

Nowadays, online auctions have become the most successful business model in the electronic marketplace. To the best of the authors' knowledge, no other work has been devoted to the prediction of closing price and duration of Business-to-Business (B2B) English reverse online auctions in which goods or service providers compete with each other to win contracts by lowering offering prices with each bid, which is conducted on a virtual platform hosted on the Internet. This research designs and proposes a new methodology to predict closing prices and duration within the first few bids of the corresponding auctions based on real time bidding information rather than static auction information. In this article, we employ real time information and prediction rules to forecast the behavior of live auctions. This contrasts with the static prediction approach that takes into consideration only information available at the beginning of an auction such as products, item features, or the seller's reputation. This simulation is based on discretized auction data derived from a B2B online auction marketplace over a two-year period. Three measurements including accuracy, coverage, and benefit are used to evaluate the methodology. Results show that after observing the first 4 bids, this methodology can predict closing prices and duration with 84.6 and 71.9% accuracy, respectively.

Image-based appraisal of real estate properties

Q. You, R. Pang, L. Cao, and J. Luo 2017

Real estate appraisal, which is the process of estimating the price for real estate properties, is crucial for both buyers and sellers as the basis for negotiation and transaction. Traditionally, the repeat sales model has been widely adopted to estimate real estate prices. However, it depends on the design and calculation of a complex economic-related index, which is challenging to estimate accurately. Today, real estate brokers provide easy access to detailed online information on real estate properties to their clients.

CHAPTER – 3

SYSTEM ANALYSIS

3.1 Existing Systems

In the existing system, buying and selling pre-owned products is mostly carried out through traditional methods such as local markets, newspaper advertisements, or informal social media groups. These methods lack a centralized platform to systematically manage product listings, user information, and feedback. Communication between buyers and sellers is unstructured, and there is limited transparency regarding product details, pricing comparison and seller credibility. Additionally, manual record-keeping makes it difficult to track transactions, manage product availability, and maintain user data efficiently.

Traditional approaches such as local markets require physical presence, making the process time-consuming and geographically restricted. Buyers and sellers need to meet in person, which may not always be convenient. Similarly, newspaper advertisements and classified listings have limited reach and do not provide real-time updates, reducing their effectiveness in today's fast-paced digital environment.

Overall, the existing system is fragmented, inefficient, and lacks the necessary tools to support a reliable and user-friendly second-hand trading environment.

3.1.1 Disadvantages of Existing Systems

1. No centralized platform to manage products and users
2. Limited search and filtering options
3. Lack of proper product categorization
4. No proper data storage and management system.
5. Lack of secure user authentication, leading to trust and security issues.
6. Time-consuming process due to manual and offline interactions.
7. High chances of fraud, fake listings, and misleading product information.

3.2 Proposed System

The proposed system is a web-based Pre-Owned Merchandise application developed using Python and MySQL that provides a centralized platform for buying and selling second-hand products. It allows users to register, log in, list products with images, search products based on category, price range, and location, and provide ratings and feedback. The admin can monitor registered users, available products, and user feedback to ensure proper management of the platform. This system automates product management, enhances transparency, and simplifies communication between buyers and sellers.

In addition to basic functionalities, the system ensures proper management of product information through a structured database. Sellers can upload detailed product descriptions along with images, which improves product visibility and helps buyers make informed decisions. The system supports multiple product categories, enabling better classification and easy navigation for users. Buyers can search for products using various filters, making the process faster and more convenient.

The proposed system also incorporates role-based access, where administrators can monitor and manage users, products, and feedback effectively. This helps maintain discipline and control within the platform. Furthermore, the system includes validation mechanisms to ensure that the data entered by users is accurate and consistent, reducing errors and improving reliability.

Overall, the proposed system provides a structured, secure, and user-friendly environment for second-hand trading. It improves efficiency, enhances transparency, and creates a reliable marketplace where users can conveniently buy and sell pre-owned products while ensuring better utilization of resources.

3.2.1 Advantages of Proposed System

1. Centralized and organized platform
2. Easy product listing with image upload
3. Advanced search and filtering options
4. It can support image storage and retrieval mechanisms for better product visualization.
5. It allows multi-category product handling, enabling better organization of different types of goods.

3.3 Feasibility study of the Proposed System

The feasibility study evaluates whether the proposed system is practical and beneficial for implementation. It considers technical, economic, and operational aspects to ensure the system can be successfully developed and used.

Technical Feasibility:

The system is technically feasible as it uses widely available technologies like Python and MySQL. These tools are easy to use, reliable, and supported on most platforms. The system does not require advanced hardware and can run on basic configurations. Its modular design also allows easy maintenance and future enhancements.

Economic Feasibility:

The project is cost-effective since it uses open-source technologies, eliminating licensing costs. It requires minimal hardware and maintenance expenses. The overall development cost is low, and the benefits such as timesaving, improved efficiency, and better resource utilization make it economically viable.

Operational Feasibility:

The system is user-friendly and easy to operate. Users can register, list products, and search for items without requiring technical knowledge. The platform simplifies buying and selling processes, while the admin module ensures proper monitoring and management.

Social Feasibility:

The system promotes reuse of products, reducing waste and supporting environmental sustainability. It also helps users, especially students, access affordable goods.

Conclusion:

Thus, the proposed system is technically, economically, and operationally feasible, making it suitable for implementation.

3.4 Software Requirement Specification

A Software Requirements Specification (SRS) – a requirements specification for a software system is a complete description of the behavior of a system to be developed. It includes a set of use cases that describe all the interactions the users will have with the software. In addition to use cases, the SRS also contains non-functional requirements. Non-functional requirements are requirements which impose constraints on the design or implementation (such as performance engineering requirements, quality standards, or design constraints).

System requirements specification: A structured collection of information that embodies the requirements of a system. A business analyst, sometimes titled system analyst, is responsible for analyzing the business needs of their clients and stakeholders to help identify business problems and propose solutions. Within the systems development lifecycle domain, the BA typically performs a liaison function between the business side of an enterprise and the information technology department or external service providers.

Projects are subject to three sorts of requirements:

- Business requirements describe in business terms what must be delivered or accomplished to provide value.
- Product requirements describe properties of a system or product (which could be one of several ways to accomplish a set of business requirements.)

Process requirements describe activities performed by the developing organization. For instance, process requirements could specify. Preliminary investigation examines project feasibility, the likelihood the system will be useful to the organization. The main objective of the feasibility study is to test the Technical, Operational and Economical feasibility for adding new modules and debugging old running system. All system is feasible if they are unlimited resources and infinite time

3.4.1 Software Requirements

- Operating System - Windows 10/above
- Programming Language - Python 3.7.0

3.4.2 Hardware Requirements

1. Speed - 1.1 GHz
2. RAM - 4GB (min)
3. Hard Disk - 500GB (min)
4. Processor - Intel I3 (min)

CHAPTER - 4

SYSTEM DESIGN

4.1 Input Design

Input design is the process of defining how data is entered into the system. The main objective is to ensure that the data entered is accurate, complete, and user-friendly. In this system, input is provided through structured web forms designed with proper validation techniques to minimize errors.

The system accepts inputs such as user registration details, login credentials, and product information. Each input field is carefully designed with constraints like required fields, data type checks, and length restrictions. For example, email fields are validated for proper format, passwords are secured, and product price fields accept only numeric values.

User-friendly interfaces are used to make data entry simple and efficient. Dropdown menus are provided for selecting product categories, and text fields are used for entering product descriptions. Proper error messages are displayed when invalid data is entered, guiding users to correct mistakes.

Overall, the input design ensures data integrity, reduces redundancy, and improves the efficiency of the system.

4.2 Output Design

Output design refers to the way the processed data is presented to users. The goal is to display information in a clear, organized, and meaningful format so that users can easily understand and make decisions.

In this system, outputs are displayed through web pages such as product listings, search results, and user profiles. The product details are shown with relevant information like product name, price, category, description, and images. The system ensures that outputs are properly formatted and structured for better readability.

Search results are displayed based on user queries with relevant filtering options. Feedback and messages are also shown clearly to enhance user interaction. The interface is designed to be responsive and visually appealing, ensuring a smooth user experience across different devices.

Overall, the output design improves usability by presenting accurate and well-organized information to users.

4.3 Algorithm

1. Start
2. User registers or logs into the system
3. If user is a seller, enter product details and upload
4. Store product data in database
5. If user is a buyer, search for products
6. Display matching products
7. Buyer views product details
8. Buyer contacts seller and completes transaction
9. User provides feedback
10. Stop

4.4 Flow Chart

Start

↓

User Login/Register

↓

Is User Seller or Buyer?

Seller

↓

Add Product

↓

Store in DB

Buyer

↓

Search Product

↓

View Results

↓

View Details

↓

Contact Seller

↓

Feedback

↓

End

4.5 UML Diagram

The Unified Modelling Language allows the software engineer to express an analysis model using the modelling notation that is governed by a set of syntactic semantic and pragmatic rules.

A UML system is represented using five different views that describe the system from distinctly different perspective. Each view is defined by a set of diagrams, which is as follows.

User Model View

This view represents the system from the user's perspective.

The analysis representation describes a usage scenario from the end-user's perspective.

Structural Model view

In this model the data and functionality arrived from inside the system.

This model view models the static structures.

Behavioral Model View

It represents the dynamic of behavioral as parts of the system, depicting the interactions of collection between various structural elements described in the user model and structural model view.

Implementation Model View

In this the structural and behavioral as parts of the system are represented as they are to be built.

Environmental Model View

In these the structural and behavioral aspects of the environment in which the system is to be implemented are represented.

4.5.1 Class Diagram

The class diagram is the main building block of object-oriented modelling. It is used both for general conceptual modelling of the systematic of the application, and for detailed modelling translating the models into programming code. Class diagrams can also be used for data modelling. The classes in a class diagram represent both the main objects, interactions in the application and the classes to be programmed. In the diagram, classes are represented with boxes which contain three parts:

- The upper part holds the name of the class.
- The middle part contains the attributes of the class.
- The bottom part gives the methods or operations the class can take or undertake.

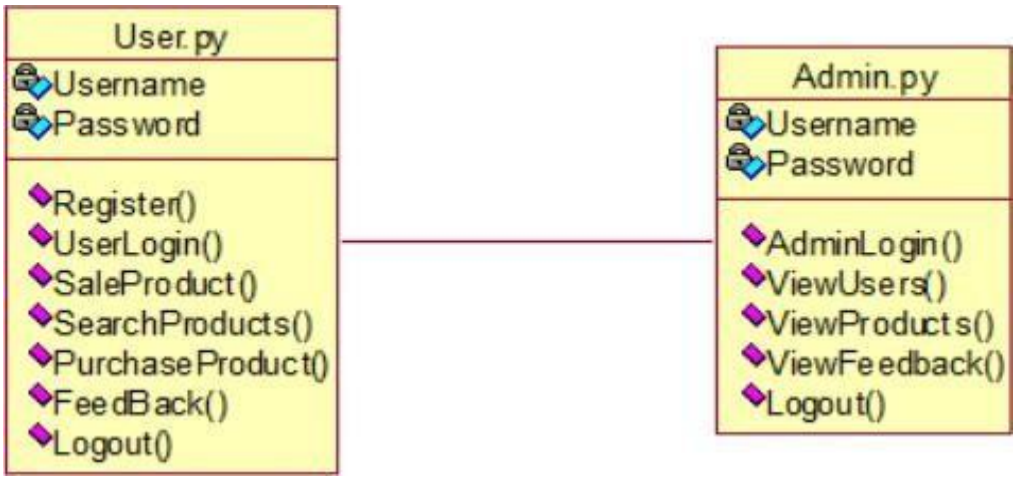


Fig: 4.5.1 Class Diagram

4.5.2 Use Case Diagram

A use case diagram at its simplest is a representation of a user's interaction with the system and depicting the specifications of a use case. A use case diagram can portray the different types of users of a system and the various ways that they interact with the system. This type of diagram is typically used in conjunction with the textual use case and will often be accompanied by other types of diagrams as well.

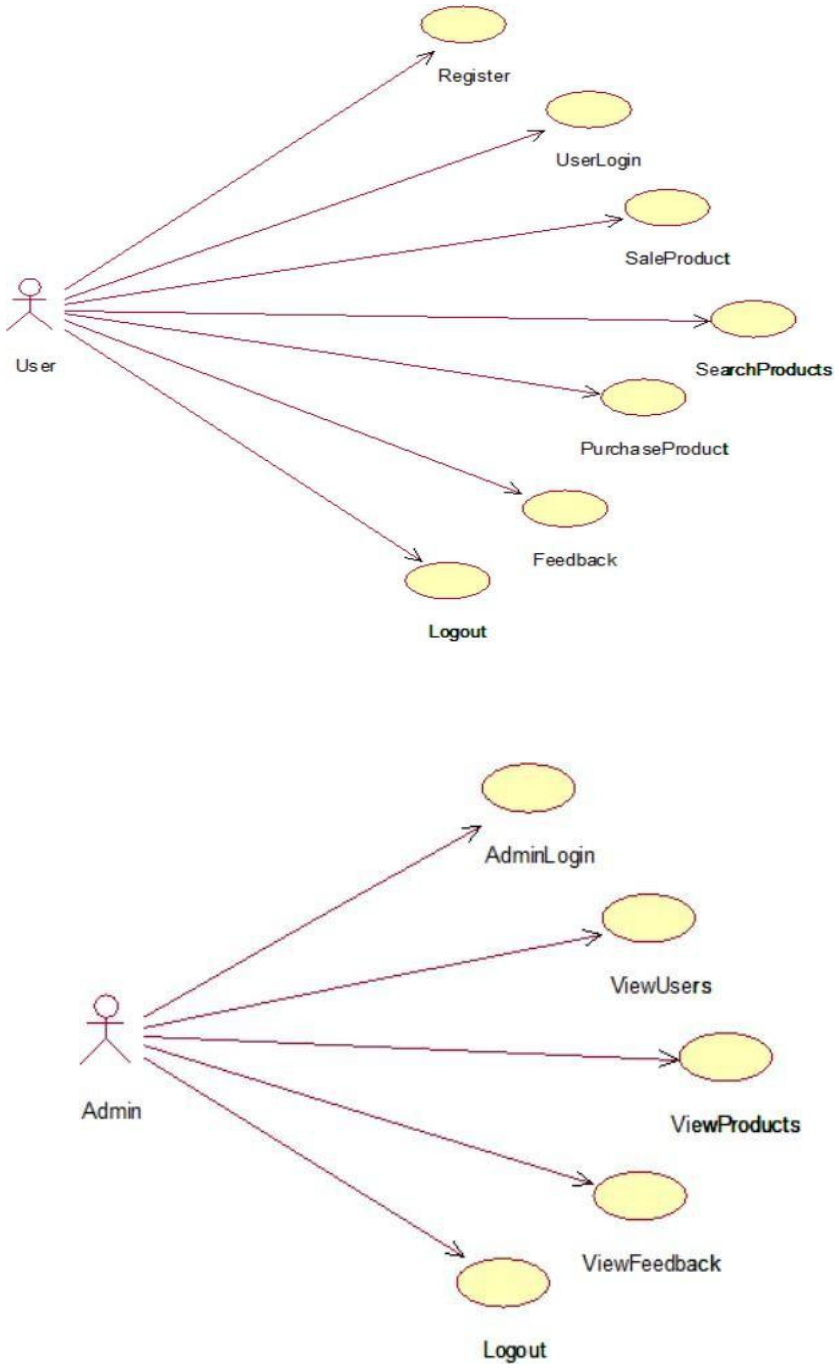


Fig: 4.5.2 Use Case Diagram

4.5.3 Sequence diagram

A sequence diagram is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. A sequence diagram shows object interactions arranged in time sequence. It depicts the objects and classes involved in the scenario and the sequence of messages exchanged between the objects needed to carry out the functionality of the scenario. Sequence diagrams are typically associated with use case realizations in the Logical View of the system under development. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.

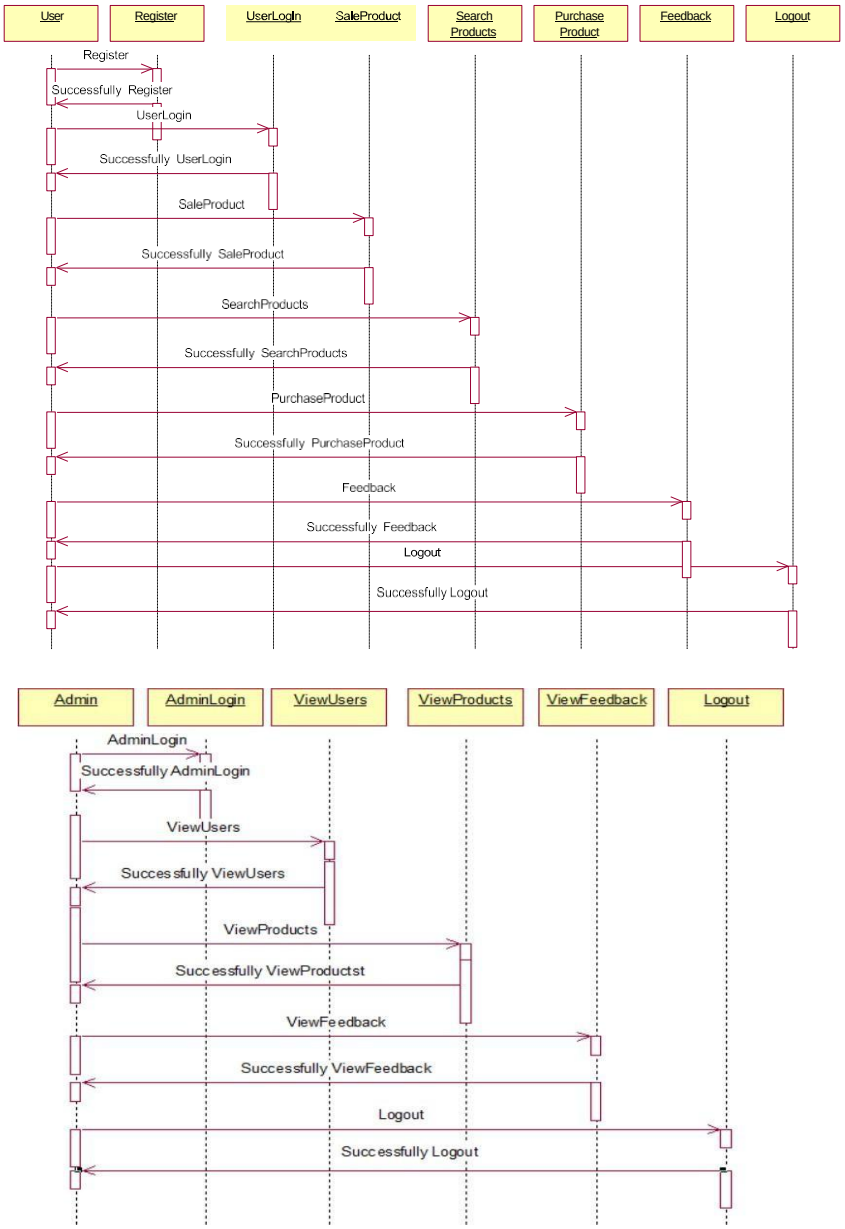


Fig: 4.5.3 Sequence Diagram

4.5.4 Collaboration diagram

A collaboration diagram describes interactions among objects in terms of sequenced messages. Collaboration diagrams represent a combination of information taken from class, sequence, and use case diagrams describing both the static structure and dynamic behaviour of a system.

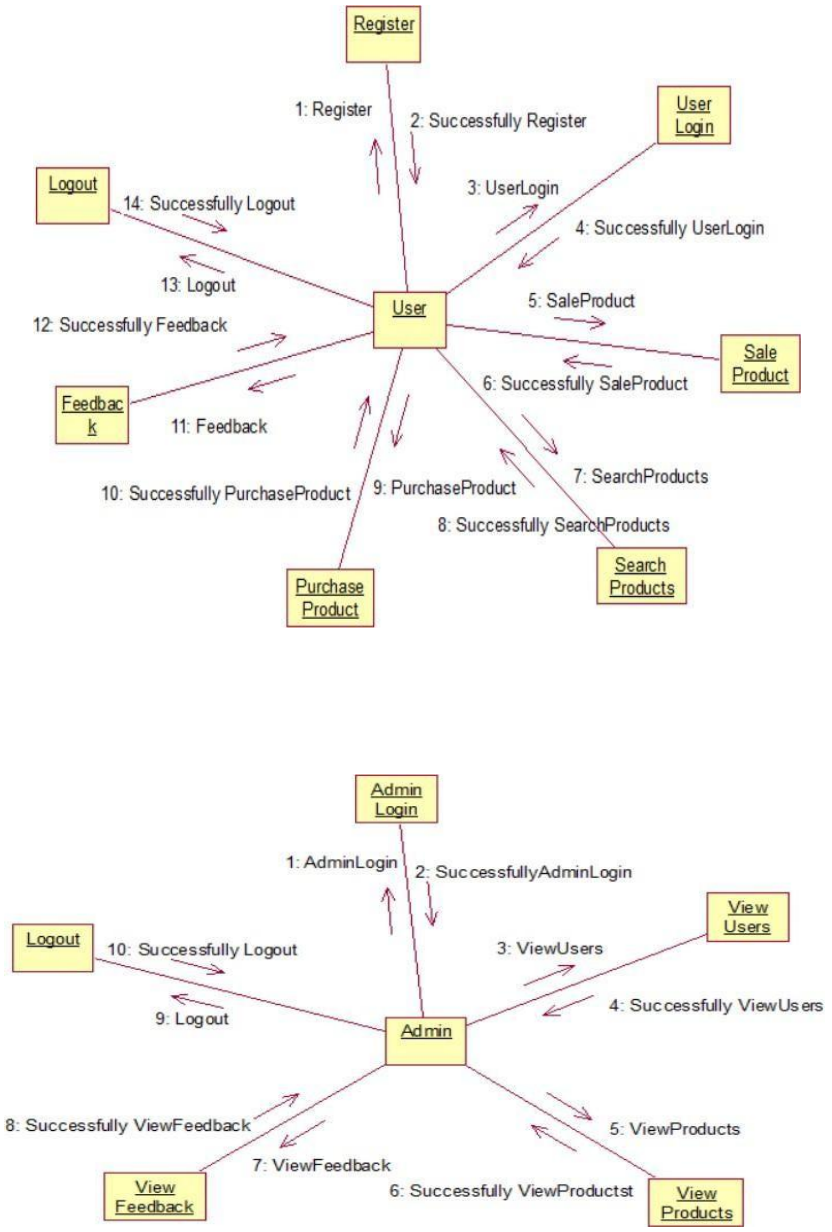


Fig: 4.5.4 Collaboration Diagram

4.5.5 Component Diagram

In the Unified Modelling Language, a component diagram depicts how components are wired together to form larger components and or software systems. They are used to illustrate the structure of arbitrarily complex systems.

Components are wired together by using an assembly connector to connect the required interface of one component with the provided interface of another component. This illustrates the service consumer - service provider relationship between the two components.

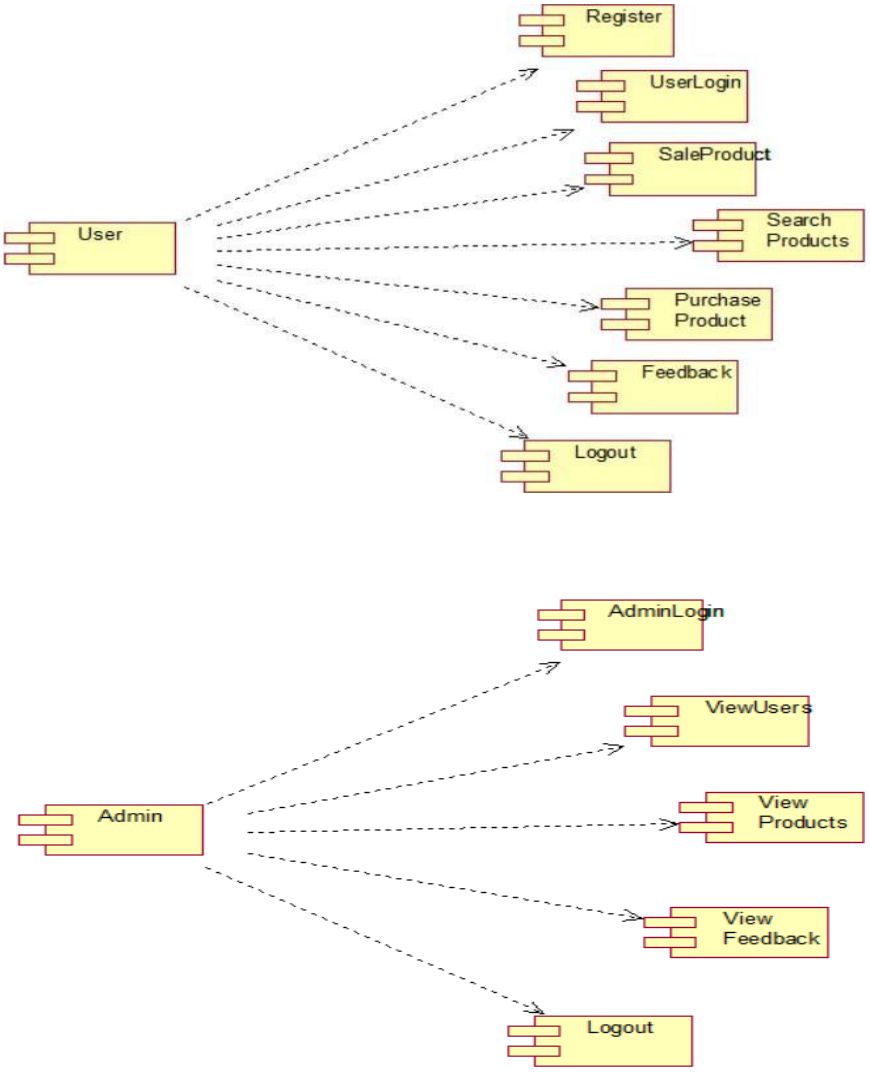


Fig: 4.5.5 Component Diagram

4.5.6 Deployment Diagram

A deployment diagram in the Unified Modelling Language models the *physical* deployment of artifacts on nodes. To describe a web site, for example, a deployment diagram would show what hardware components ("nodes") exist (e.g., a web server, an application server, and a database server), what software components ("artifacts") run on each node (e.g., web application, database), and how the different pieces are connected (e.g. JDBC, REST, RMI).

The nodes appear as boxes, and the artifacts allocated to each node appear as rectangles within the boxes. Nodes may have sub nodes, which appear as nested boxes. A single node in a deployment diagram may conceptually represent multiple physical nodes, such as a cluster of database servers.

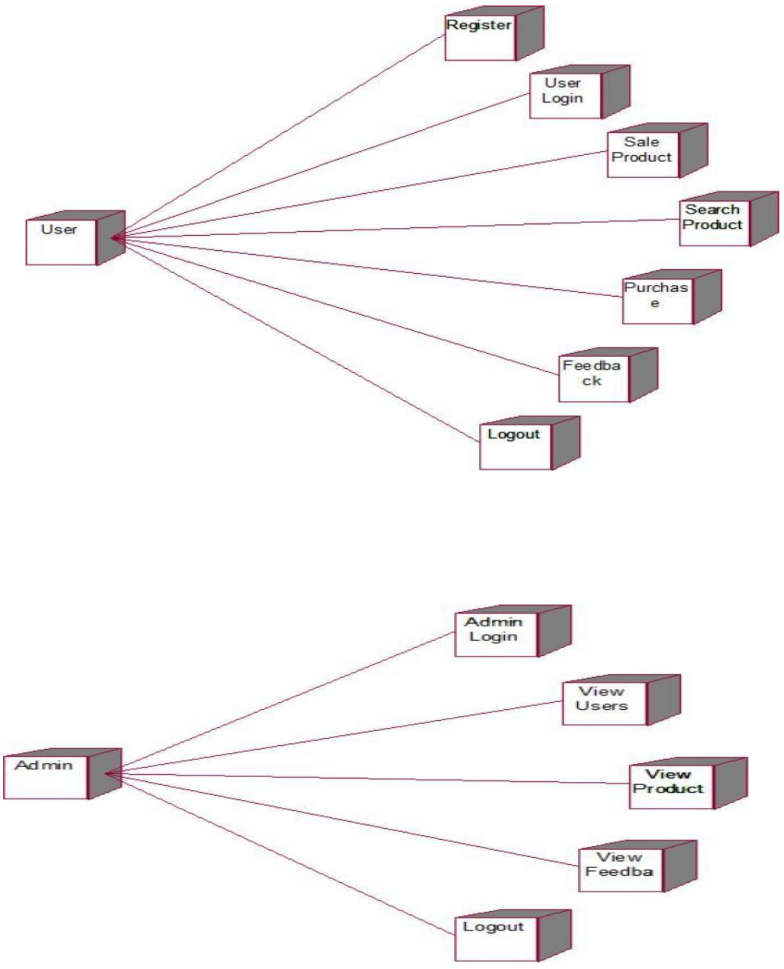


Fig: 4.5.6 Deployment Diagram

4.5.7 Activity Diagram

Activity diagram is another important diagram in UML to describe dynamic aspects of the system. It is basically a flow chart to represent the flow form one activity to another activity. The activity can be described as an operation of the system. So, the control flow is drawn from one operation to another. This flow can be sequential, branched or concurrent

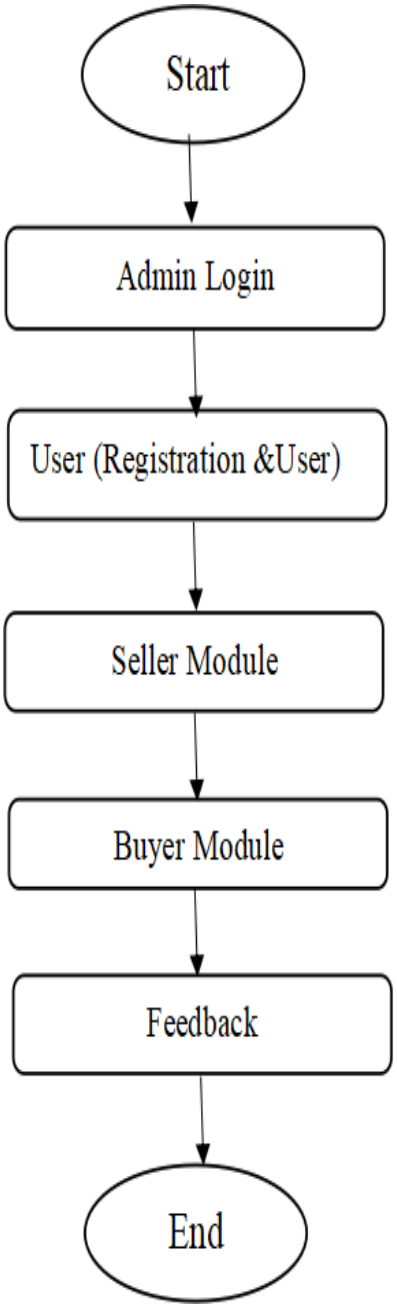


Fig: 4.5.7 Activity Diagram

4.5.8 Data Flow Diagram

Data flow diagrams illustrate how data is processed by a system in terms of inputs and outputs. Data flow diagrams can be used to provide a clear representation of any business function. The technique starts with an overall picture of the business and continues by analysing each of the functional areas of interest. This analysis can be carried out in precisely the level of detail required. The technique exploits a method called top-down expansion to conduct the analysis in a targeted way.

As the name suggests, Data Flow Diagram (DFD) is an illustration that explicates the passage of information in a process. A DFD can be easily drawn using simple symbols. Additionally, complicated processes can be easily automated by creating DFDs using easy-to-use, free downloadable diagramming tools. A DFD is a model for constructing and analyzing information processes. DFD illustrates the flow of information in a process depending upon the inputs and outputs. A DFD can also be referred to as a Process Model. A DFD demonstrates business or technical process with the support of the outside data saved, plus the data flowing from the process to another and the end results.

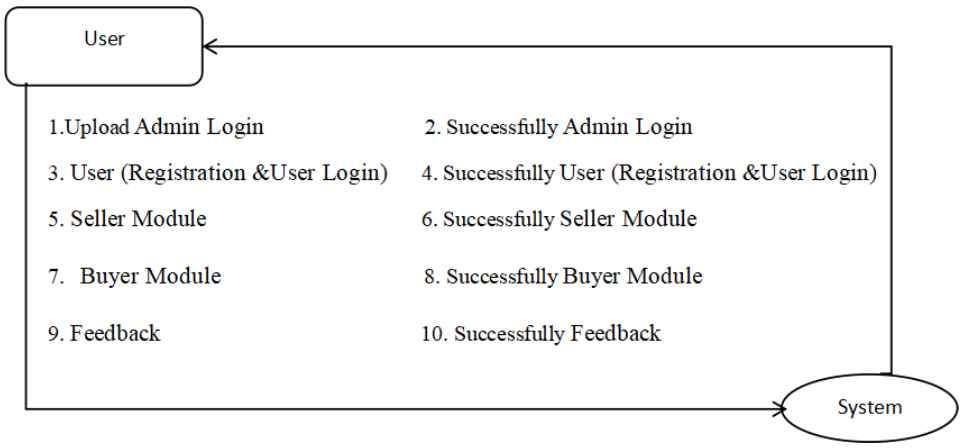


Fig: 4.5.8 Data Flow Diagram

4.5.9 System Architecture

1. A centralized system where all modules are connected to a central database. This database stores and manages all the information such as user details, product listings, and feedback.
2. The User Module allows new users to register and existing users to log in to the system. Once logged in, users can act as buyers or sellers.
3. The Seller Module enables users to add products and manage their listings. Sellers can upload product details, making them available in the system for buyers.
4. The Buyer Module allows users to search for products and contact sellers. It helps users easily find and interact with available items.
5. The Admin Module is responsible for managing the entire system. The admin can monitor users, manage product listings, and view feedback to ensure smooth functioning.
6. The Feedback & Rating Module allows users to provide reviews and ratings based on their experience, improving transparency and trust in the system.
7. All these modules interact with the central database, ensuring that data is stored securely and accessed efficiently.

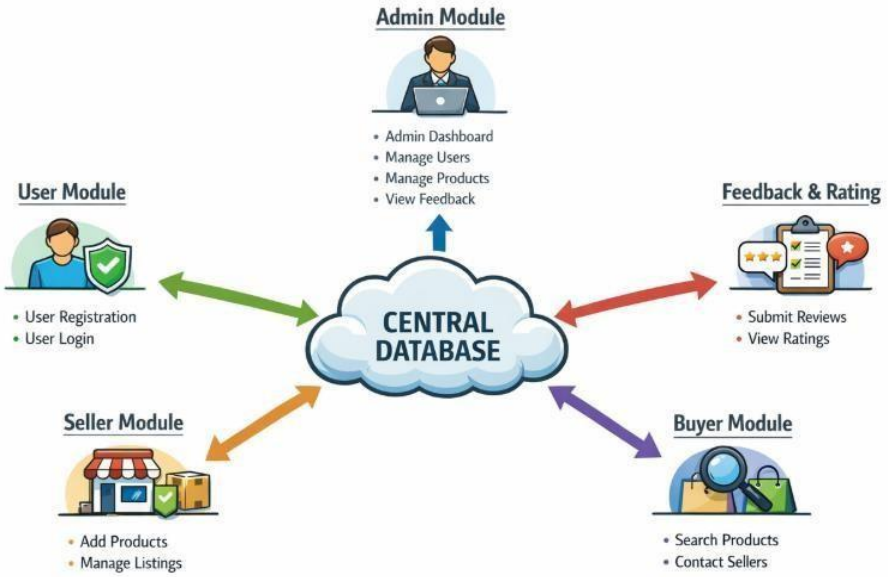


Fig: 4.5.9 System Architecture

CHAPTER – 5

IMPLEMENTATION

5.1 Methods of Implementation

To implement this project, we have designed the following modules:

1. Admin

The Admin Module manages and monitors the entire system. After logging in with predefined credentials, the admin can view all registered users, available products, and feedback with ratings. This module ensures proper supervision and maintenance of the platform.

2. User (Registration &User Login)

The User Module allows new users to register by providing personal details and create an account in the system. Registered users can log in securely using their username and password. After login, users can act as sellers or buyers based on their needs.

3. Seller Module

The Seller Module enables logged-in users to add product details such as name, category, price, description, location, and upload images. Sellers can list multiple products for sale. This module manages product creation and storage in the database.

4. Buyer Module

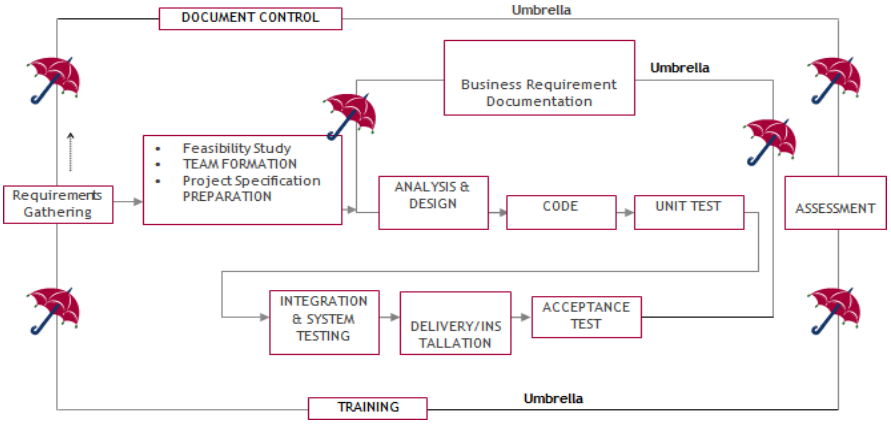
The Buyer Module allows users to search for products based on category, price range, and location. Buyers can view detailed product information along with images and seller contact details. This helps them finalize purchase deals offline.

5. Feedback

The Feedback allows buyers to provide ratings and comments for products. Submitted feedback is stored in the database and can be viewed by the admin. This module improves transparency and builds trust among users.

PROCESS MODEL USED WITH JUSTIFICATION

SDLC (Umbrella Model):



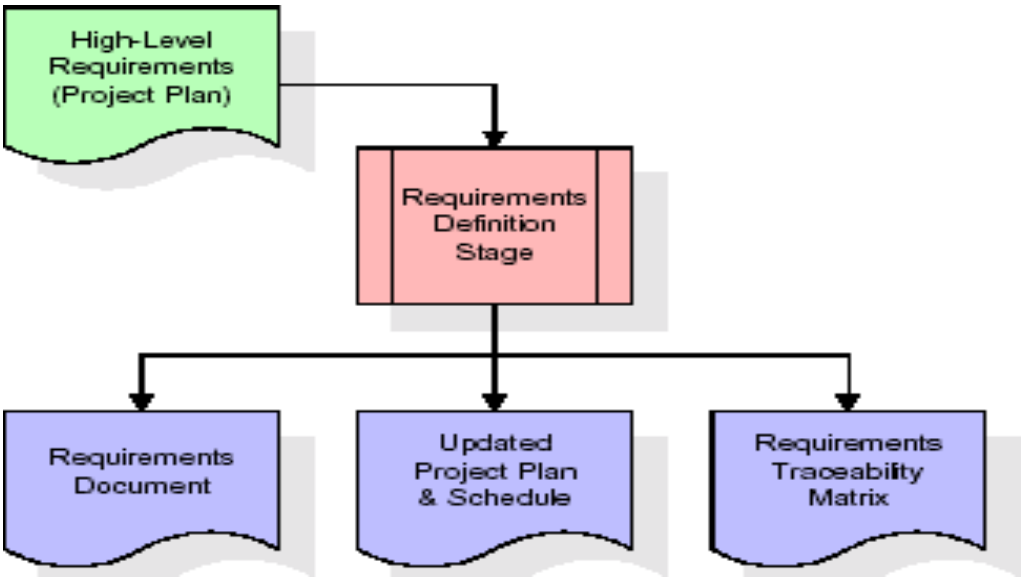
SDLC is nothing but Software Development Life Cycle. It is a standard which is used by software industry to develop good software.

Stages in SDLC:

- Requirement Gathering
- Analysis
- Designing
- Coding
- Testing
- Maintenance

Requirements Gathering stage:

The requirements gathering process takes as its input the goals identified in the high-level requirements section of the project plan. Each goal will be refined into a set of one or more requirements. These requirements define the major functions of the intended application, define operational data areas and reference data areas, and define the initial data entities. Major functions include critical processes to be managed, as well as mission critical inputs, outputs and reports. A user class hierarchy is developed and associated with these major functions, data areas, and data entities. Each of these definitions is termed a Requirement. Requirements are identified by unique requirement identifiers and, at minimum, contain a requirement title and textual description.



These requirements are fully described in the primary deliverables for this stage: the Requirements Document and the Requirements Traceability Matrix (RTM). The requirements document contains complete descriptions of each requirement, including diagrams and references to external documents as necessary. Note that detailed listings of database tables and fields are *not* included in the requirements document.

The title of each requirement is also placed into the first version of the RTM, along with the title of each goal from the project plan. The purpose of the RTM is to show that the product components developed during each stage of the software development lifecycle are formally connected to the components developed in prior stages.

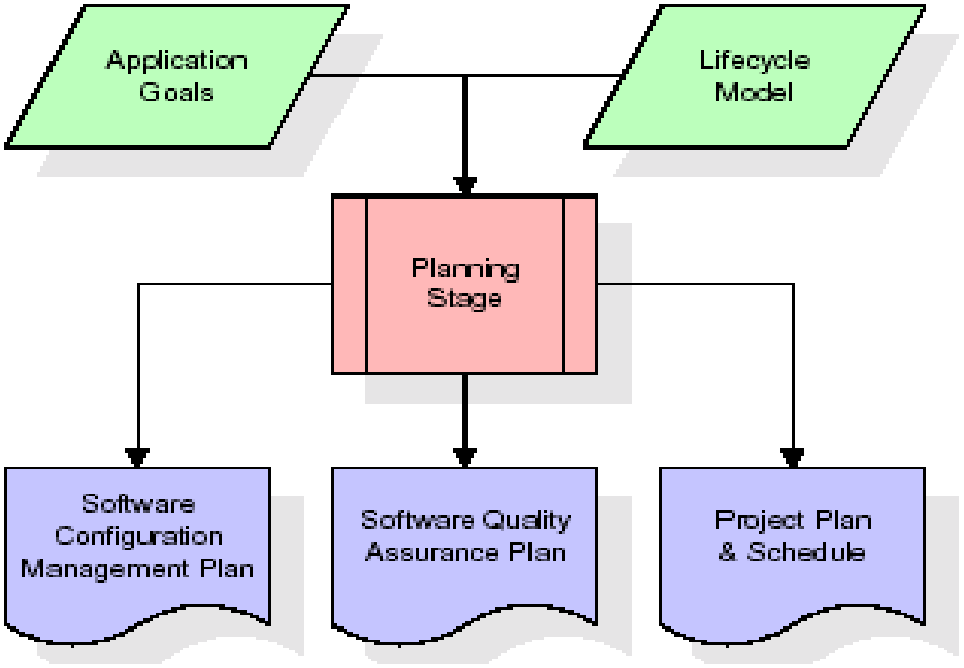
In the requirements stage, the RTM consists of a list of high-level requirements, or goals, by title, with a listing of associated requirements for each goal, listed by requirement title. In this hierarchical listing, the RTM shows that each requirement developed during this stage is formally linked to a specific product goal. In this format, each requirement can be traced to a specific product goal, hence the term requirements traceability.

The outputs of the requirements definition stage include the requirements document, the RTM, and an updated project plan.

- Feasibility study is all about identification of problems in a project.
- No. of staff required to handle a project is represented as Team Formation, in this case only modules are individual tasks will be assigned to employees who are working for that project.
- Project Specifications are all about representing of various possible inputs submitting to the server and corresponding outputs along with reports maintained by administrator.

Analysis Stage:

The planning stage establishes a bird's eye view of the intended software product, and uses this to establish the basic project structure, evaluate feasibility and risks associated with the project, and describe appropriate management and technical approaches.

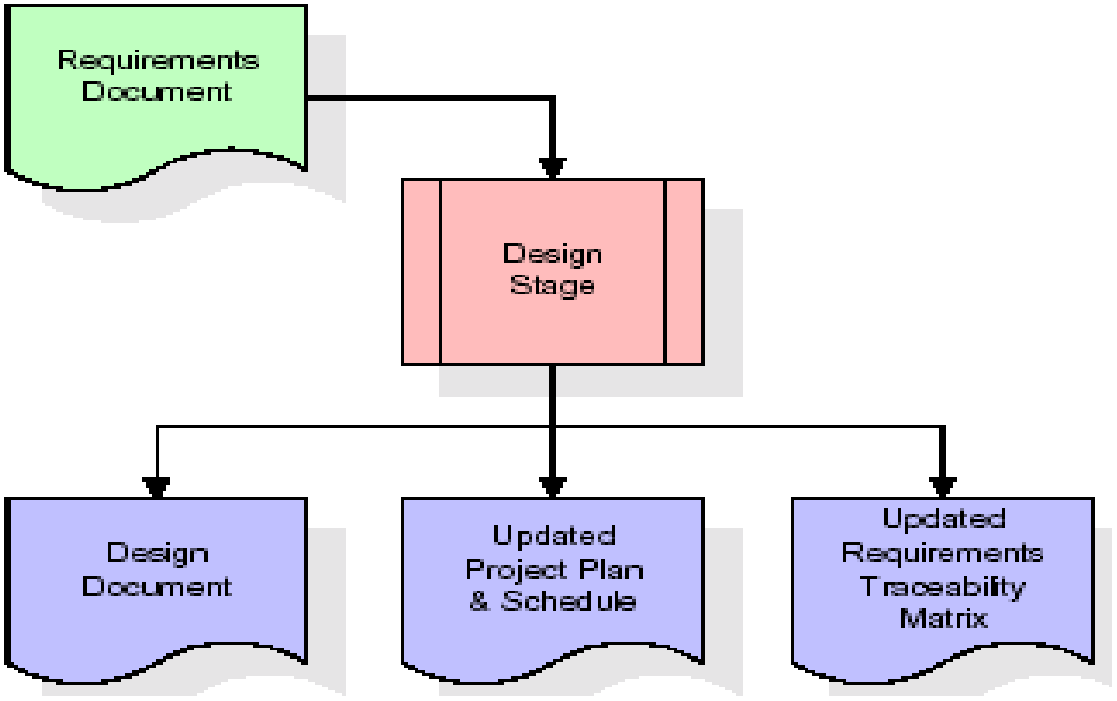


The most critical section of the project plan is a listing of high-level product requirements, also referred to as goals. All the software product requirements to be developed during the requirements definition stage flow from one or more of these goals. The minimum information for each goal consists of a title and textual description, although additional information and references to external documents may be included. The outputs of the project planning stage are the configuration management plan, the quality assurance plan, and the project plan and schedule, with a detailed listing of scheduled activities for the upcoming Requirements stage, and high-level estimates of effort for the out stages.

Designing Stage:

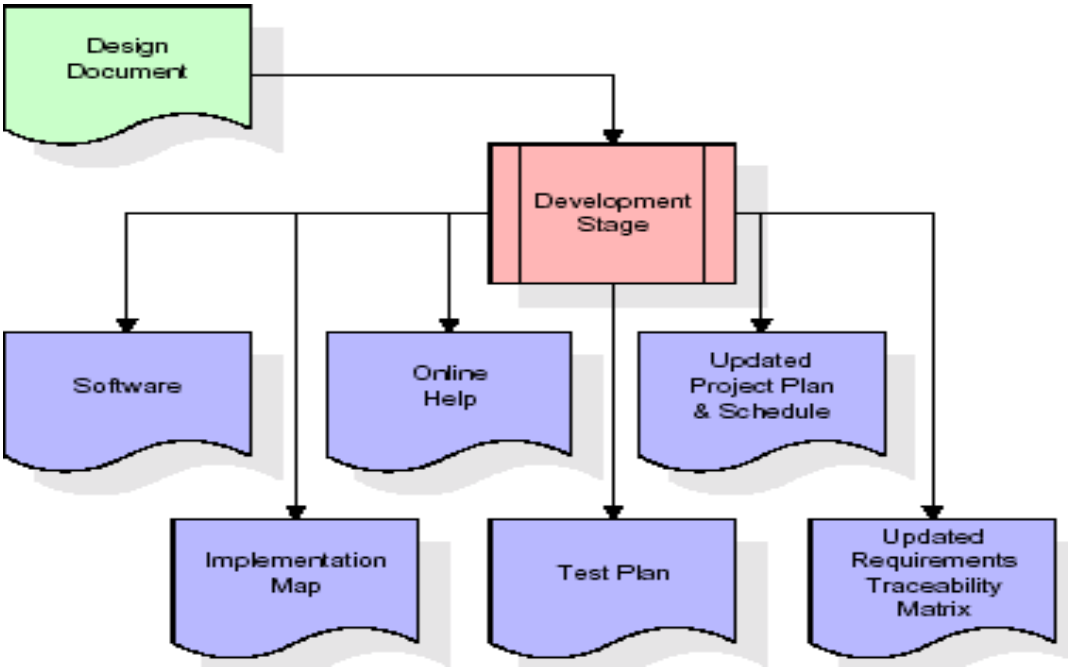
The design stage takes as its initial input the requirements identified in the approved requirements document. For each requirement, a set of one or more design elements will be produced because of interviews, workshops, and/or prototype efforts. Design elements describe the desired software features in detail, and generally include functional hierarchy diagrams, screen layout diagrams, tables of business rules, business process diagrams, pseudo code, and a complete entity-relationship diagram with a full data dictionary. These design elements are intended to describe the software in sufficient detail that skilled programmers may develop the software with minimal additional input.

When the design document is finalized and accepted, the RTM is updated to show that each design element is formally associated with a specific requirement. The outputs of the design stage are the design document, an updated RTM, and an updated project plan.



Development (Coding) Stage:

The development stage takes as its primary input the design elements described in the approved design document. For each design element, a set of one or more software artifacts will be produced. Software artifacts include but are not limited to menus, dialogs, and data management forms, data reporting formats, and specialized procedures and functions. Appropriate test cases will be developed for each set of functionally related software artifacts, and an online help system will be developed to guide users in their interactions with the software.



The RTM will be updated to show that each developed artifact is linked to a specific design element, and that each developed artifact has one or more corresponding test case items. At this point, the RTM is in its final configuration. The outputs of the development stage include a fully functional set of software that satisfies the requirements and design elements previously documented, an online help system that describes the operation of the software, an implementation map that identifies the primary code entry points for all major system functions, a test plan that describes the test cases to be used to validate the correctness and completeness of the software, an updated RTM, and an updated project plan.

5.2 Software Specification

5.2.1 Python

- One of the most popular languages is Python. Guido van Rossum released this language in 1991. Python is available on the Mac, Windows, and Raspberry Pi operating systems. The syntax of Python is simple and identical to that of English. When compared to Python, it was seen that the other language requires a few extra lines.
- It is an interpreter-based language because code may be run line by line after it has been written. This implies that rapid prototyping is possible across all platforms. Python is a big language with a free, binary-distributed interpreter standard library.
- It is inferior to maintenance that is conducted and is straightforward to learn. It is an object-oriented, interpreted programming language. It supports several different programming paradigms in addition to object-oriented programming, including functional and procedural programming.
- It supports several different programming paradigms in addition to object-oriented programming, including practical and procedural programming. Python is mighty while maintaining a relatively straightforward syntax. Classes, highly dynamic data types, modules, and exceptions are covered. Python can also be utilized by programmed that require programmable interfaces as an external language.

Python IDLE

- ❖ Python IDLE offers a full-fledged file editor, which gives you the ability to write and execute Python programs from within this program. The built-in file editor also includes several features, like code completion and automatic indentation, that will speed up your coding workflow.
- ❖ Guido Van Rossum named Python after the British comedy group Monty Python while the name IDLE was chosen to pay tribute to Eric Idle, who was one of the Monty Python's founding members. IDLE comes bundled with the default implementation of the Python language since the 01.5. 2b1 release
- ❖ IDLE is used to execute statements similar to Python Shell. IDLE is used to create, modify, and execute Python code. IDLE provides a fully-featured text editor to write Python scripts and provides features like syntax highlighting, auto-completion, and smart indent.

- ❖ IDLE has two modes: interactive and script. We wrote our first program, “Hello, World!” in interactive mode. Interactive mode immediately returns the results of commands you enter the shell. In script mode, you will write a script and then run it.
- ❖ The IDE Python IDLE is a good place to start as it helps you become familiar with the way Python works and understand its syntax. This IDE is good to start programming in Python due to its great debugger, but once you are fluent and start developing projects it is necessary to jump to another, more complete IDE.
- ❖ Python IDLE (Integrated Development and Learning Environment) is an interactive development environment included with the Python programming language. It provides a convenient way to write, execute, and debug Python code.

When you install Python, IDLE is typically installed along with it. To open IDLE, you can follow these steps:

- o Open the command prompt (Windows) or terminal (macOS/Linux).
- o Type "idle" and press Enter. Alternatively, you can specify the version with "idle3" or "idle2" for Python 3 or Python 2, respectively.
- o Once IDLE is launched, you will see the Python shell, which is an interactive environment where you can type and execute Python code directly.

Here are some features and functionalities provided by Python IDLE:

- Editor: IDLE includes a text editor where you can write your Python code. It offers syntax highlighting, automatic indentation, and code completion to enhance your coding experience.
- Interactive Shell: The Python shell in IDLE allows you to execute Python code interactively. You can type commands, statements, or function calls directly in the shell, and Python will execute them immediately.
- Debugging: IDLE provides basic debugging capabilities to help you find and fix errors in your code. You can set breakpoints, step through code, inspect variables, and track the program's execution.
- Python Help: IDLE provides access to the Python documentation and built-in help. You can access the help menu to find information about Python modules, functions, classes, and more.
- Script Execution: In addition to the interactive shell, IDLE allows you to run Python scripts stored in files. You can write your code in the editor and execute it as a script to see the output or interact with the program.

Installation

To install Python on your computer, follow these basic steps:

Step 1: Visit the Python website Go to the official Python website at https://www.python.org/.

Step 2: Select the operating system Choose the appropriate installer for your operating system. Python supports Windows, macOS, and various Linux distributions. Make sure to select the correct version that matches your operating system.

Step 3: Check which version of Python is installed; if the 3.7.0 version is not there, uninstall it through the control panel and

Step 4: Install Python 3.7.0 using Cmd.

Step 5: Install all libraries that required to run the project

Step 6: Run

5.2.2 Features of Python:

1. **Easy:** Because Python is a more accessible and straightforward language, Python programming is easier to learn.
2. **Interpreted language:** Python is an interpreted language; therefore it can be used to examine the code line by line and provide results.
3. **Open Source:** Python is a free online programming language since it is open source.
4. **Portable:** Python is portable because the same code may be used on several computer standard
5. **Libraries:** Python offers a sizable library that we may utilize to create applications quickly.
6. **GUI:** It stands for GUI (Graphical User Interface)
7. **Dynamical typed:** Python is a dynamically typed language; therefore, the type of the value will be determined at runtime.

5.2.3 Libraries in Python

In Python, libraries (also referred to as modules or packages) are collections of pre-written code that provide additional functionality and tools to extend the capabilities of the Python language. Libraries contain reusable code that developers can leverage to perform specific tasks without having to write everything from scratch.

Python libraries are designed to solve common problems, such as handling data, performing mathematical operations, interacting with databases, working with files, implementing networking protocols, creating graphical user interfaces (GUIs), and much more. They provide ready-to-use functions, classes, and methods that simplify complex operations and save development time.

Libraries in Python offer various advantages:

- Code Reusability
- Efficiency
- Collaboration
- Domain-Specific Functionality
- To use a Python library, you need to install it first.

There are some libraries following:

Django

Django is a high-level Python web framework used for building secure and scalable web applications. It provides built-in tools and features that help developers create web applications quickly and efficiently. Django follows the MVT (Model-View-Template) architectural pattern and is released under the BSD license. It is widely used for developing dynamic websites and web-based applications. Django is a Python framework that provides fast, secure, and maintainable web development. It is designed to encourage rapid development and clean, pragmatic design. Django handles much of the complexity of web development, allowing developers to focus on writing application logic.

- It provides built-in features like authentication, admin panel, ORM, and security.
- It follows the "Don't Repeat Yourself (DRY)" principle.
- It includes an automatic admin interface for managing database records.
- It has a powerful Object-Relational Mapper (ORM) for database operations.
- It supports multiple databases such as PostgreSQL, MySQL, and SQLite.
- It was created by Adrian Holovaty and Simon Willison in 2005.

Django allows developers to build everything from simple websites to complex, data-driven web applications. It is widely used in startups and large companies because of its scalability and security features. It has a strong community and extensive documentation.

PyMYSQL

PyMySQL is a pure-Python library used to connect Python applications to MySQL databases. It allows developers to interact with MySQL databases using Python code. PyMySQL implements the Python Database API (DB-API 2.0) specification. PyMySQL is a Python package that provides an interface for connecting to and working with **MySQL** databases. It is lightweight, easy to use, and works well with frameworks like Django and Flask.

- It allows executing SQL queries such as SELECT, INSERT, UPDATE, and DELETE.
- It supports transactions and database connection handling.
- It is written entirely in Python (no external dependencies required).
- It follows Python DB-API 2.0 standards.
- It can be used as a drop-in replacement for MySQLdb in many cases.

PyMySQL helps in managing relational databases efficiently from Python applications. It is commonly used in web development and data-driven applications where MySQL is the backend database.

5.3 Sample Code:

Views.py

```
from django.shortcuts import render
from django.template import RequestContext
from django.contrib import messages
import pymysql
from django.http import HttpResponse
from django.core.files.storage import FileSystemStorage
import os
from datetime import date

global uname

def getDetails(owner):
    contact_no = "none"
    con = pymysql.connect(host='127.0.0.1',port = 3306,user = 'root', password = 'root', database
= 'preowned',charset='utf8')
    with con:
        cur = con.cursor()
        cur.execute("select contact_no from signup where username = '"+owner+"'")
        rows = cur.fetchall()
        for row in rows:
            contact_no = row[0]
            break
    return contact_no

def PurchaseProduct(request):
    if request.method == 'GET':
        global uname
        pname = request.GET.get('t1', False)
        price = request.GET.get('t2', False)
        dd = str(date.today())
```

```
        db_connection = pymysql.connect(host='127.0.0.1',port = 3306,user = 'root', password = 'root', database = 'preowned',charset='utf8')
        db_cursor = db_connection.cursor()
                                student_sql_query = "INSERT INTO purchase VALUES('"+uname+"','"+pname+"','"+price+"','"+dd+"')"
        db_cursor.execute(student_sql_query)
        db_connection.commit()
        context= {'data':'Product Purchase successfully recorded in database'}
        return render(request, 'UserScreen.html', context)


def ViewHistory(request):
    if request.method == 'GET':
        global uname
                output ='<table border=1 align=center width=100%><tr><th><font size="" color="black">Purchase Name</th><th><font size="" color="black">Product Name</th>'
            output += '<th><font size="" color="black">Product Price</th><th><font size="" color="black">Purchase Date</th>'
        output+='</tr>'
        query = "select * from purchase where purchaser_name='"+uname+"'"
          con = pymysql.connect(host='127.0.0.1',port = 3306,user = 'root', password = 'root', database = 'preowned',charset='utf8')
        with con:
            cur = con.cursor()
            cur.execute(query)
            rows = cur.fetchall()
            for row in rows:
                user = row[0]
                product = row[1]
                price = str(row[2])
                dd = row[3]
                  output += '<tr><td><font size="" color="black">'+user+'</td><td><font size="" color="black">'+product+'</td>'
                    output += '<td><font size="" color="black">'+price+'</td><td><font size="" color="black">'+dd+'</td></tr>'
```

```
        output += "</table><br/><br/><br/><br/>"
        context= {'data':output}
        return render(request, 'UserScreen.html', context)


def SearchProductsAction(request):
    if request.method == 'POST':
        global uname
        category = request.POST.get('t1', False)
        price = request.POST.get('t2', False)
        location = request.POST.get('t3', False)
        from_price = 0
        to_price = 0
        if 'Above' in price:
            from_price = 1000
            to_price = 100000
        else:
            arr = price.split("-")
            from_price = float(arr[0].strip())
            to_price = float(arr[1].strip())
                output ='<table border=1 align=center width=100%><tr><th><font size="" color="black">Owner Name</th><th><font size="" color="black">Contact No</th>'
            output += '<th><font size="" color="black">Product ID</th><th><font size="" color="black">Product Name</th>'
            output += '<th><font size="" color="black">Description</th><th><font size="" color="black">Price</th>'
        output += '<th><font size="" color="black">Product Condition</th><th><font size="" color="black">Product Category</th>'
                output += '<th><font size="" color="black">Location</th><th><font size="" color="black">Image</th>'
        output += '<th><font size="" color="black">Purchase Product</th>'
        output+='</tr>'
            query = "select * from product where category='"+category+"' and price >= '"+str(from_price)+"' and price <= '"+str(to_price)+"' and location like '%"+location+"%'"
```

```
        con = pymysql.connect(host='127.0.0.1',port = 3306,user = 'root', password = 'root', database = 'preowned',charset='utf8')
        with con:
            cur = con.cursor()
            cur.execute(query)
            rows = cur.fetchall()
            for row in rows:
                owner = row[0]
                contact = getDetails(owner)
                pid = row[1]
                pname = row[2]
                desc = row[3]
                price = row[4]
                prod_condition = row[5]
                categories = row[6]
                locations = row[7]
                image = row[8]
                 output += '<tr><td><font size="" color="black">'+owner+'</td><td><font size="" color="black">'+contact+'</td>'
                    output += '<td><font size="" color="black">'+str(pid)+'</td><td><font size="" color="black">'+pname+'</td>'
                      output += '<td><font size="" color="black">'+desc+'</td><td><font size="" color="black">'+str(price)+'</td>'
                    output += '<td><font size="" color="black">'+prod_condition+'</td><td><font size="" color="black">'+categories+'</td>'
                output += '<td><font size="" color="black">'+locations+'</td>'
               output += '<td><img src="static/photo/'+image+'" height="200" width="200"/></td>'
                                                                        output+='<td><a href=\'PurchaseProduct?t1='+str(pname)+'&t2='+str(price)+'\'><font size=3 color=red>Click Here to Purchase</font></a></td></tr>'
        output += "</table><br/><br/><br/><br/>"
        context= {'data':output}
        return render(request, 'UserScreen.html', context)
```

```
def SearchProducts(request):
    if request.method == 'GET':
        return render(request, 'SearchProducts.html', {})

def SaleProduct(request):
    if request.method == 'GET':
        return render(request, 'SaleProduct.html', {})

def SaleProductAction(request):
    if request.method == 'POST':
        global uname
        pname = request.POST.get('t1', False)
        desc = request.POST.get('t2', False)
        price = request.POST.get('t3', False)
        condition = request.POST.get('t4', False)
        category = request.POST.get('t5', False)
        location = request.POST.get('t6', False)
        filename = request.FILES['t7'].name
        myfile = request.FILES['t7'].read() #reading uploaded file from user
        pid = 1
          con = pymysql.connect(host='127.0.0.1',port = 3306,user = 'root', password = 'root',
database = 'preowned',charset='utf8')
        with con:
            cur = con.cursor()
            cur.execute("select max(product_id) from product")
            rows = cur.fetchall()
            for row in rows:
                pid = row[0]
        if pid is None:
            pid = 1
        else:
            pid += 1
        db_connection = pymysql.connect(host='127.0.0.1',port = 3306,user = 'root', password =
'root', database = 'preowned',charset='utf8')
```

```
        db_cursor = db_connection.cursor()
                                student_sql_query = "INSERT INTO product
VALUES('"+uname+"','"+str(pid)+"','"+pname+"','"+desc+"','"+price+"','"+condition+"','"+ca
tegory+"','"+location+"','"+filename+"')"
        db_cursor.execute(student_sql_query)
        db_connection.commit()
        if os.path.exists('ProductApp/static/photo/'+filename):
            os.remove('ProductApp/static/photo/'+filename)
        with open('ProductApp/static/photo/'+filename, "wb") as file:
            file.write(myfile)
        file.close()
        context= {'data':'Product details added with Product ID = '+str(pid)}
        return render(request, 'SaleProduct.html', context)

def Feedback(request):
    if request.method == 'GET':
        output = '<tr><td><font size="3" color="black">Product ID</b></td><td><select
name="t1">'
          con = pymysql.connect(host='127.0.0.1',port = 3306,user = 'root', password = 'root',
database = 'preowned',charset='utf8')
        with con:
            cur = con.cursor()
            cur.execute("select product_id from product")
            rows = cur.fetchall()
            for row in rows:
                output += '<option value="'+str(row[0])+'">'+str(row[0])+'</option>'
        output += "</select></td></tr>"
        context= {'data1': output}
        return render(request, 'Feedback.html', context)

def UserLogin(request):
    if request.method == 'GET':
       return render(request, 'UserLogin.html', {})
```

```
def index(request):
    if request.method == 'GET':
        return render(request, 'index.html', {})

def Register(request):
    if request.method == 'GET':
        return render(request, 'Register.html', {})

def FeedbackAction(request):
    if request.method == 'POST':
        global uname
        product_id = request.POST.get('t1', False)
        feedback = request.POST.get('t2', False)
        ratings = request.POST.get('t3', False)
        db_connection = pymysql.connect(host='127.0.0.1',port = 3306,user = 'root', password =
'root', database = 'preowned',charset='utf8')
        db_cursor = db_connection.cursor()
        student_sql_query = "INSERT INTO feedback(username,product_id,feedback,ratings)
VALUES('"+uname+"','"+product_id+"','"+feedback+"','"+ratings+"')"
        db_cursor.execute(student_sql_query)
        db_connection.commit()
        context= {'data':'Your feedback accepted'}
        return render(request, 'UserScreen.html', context)

def RegisterAction(request):
    if request.method == 'POST':
        username = request.POST.get('t1', False)
        password = request.POST.get('t2', False)
        contact = request.POST.get('t3', False)
        email = request.POST.get('t4', False)
        address = request.POST.get('t5', False)
        status = 'none'
        con = pymysql.connect(host='127.0.0.1',port = 3306,user = 'root', password = 'root',
database = 'preowned',charset='utf8')
```

```
        with con:
            cur = con.cursor()
            cur.execute("select username from signup where username = '"+username+"'")
            rows = cur.fetchall()
            for row in rows:
                if row[0] == email:
                    status = 'Given Username already exists'
                    break
        if status == 'none':
            db_connection = pymysql.connect(host='127.0.0.1',port = 3306,user = 'root', password
= 'root', database = 'preowned',charset='utf8')
            db_cursor = db_connection.cursor()
                                                student_sql_query = "INSERT INTO
signup(username,password,contact_no,email_id,address)
VALUES('"+username+"','"+password+"','"+contact+"','"+email+"','"+address+"')"
            db_cursor.execute(student_sql_query)
            db_connection.commit()
            print(db_cursor.rowcount, "Record Inserted")
            if db_cursor.rowcount == 1:
                status = 'Signup Process Completed'
        context= {'data':status}
        return render(request, 'Register.html', context)

def AdminLogin(request):
    if request.method == 'GET':
        return render(request, 'AdminLogin.html', {})

def AdminLoginAction(request):
    if request.method == 'POST':
        global uname
        option = 0
        username = request.POST.get('username', False)
        password = request.POST.get('password', False)
        if username == "admin" and password == "admin":
```

```
            context= {'data':'welcome '+username}
            return render(request, 'AdminScreen.html', context)
        else:
            context= {'data':'Invalid login details'}
            return render(request, 'AdminLogin.html', context)

def UserLoginAction(request):
    if request.method == 'POST':
        global uname
        option = 0
        username = request.POST.get('username', False)
        password = request.POST.get('password', False)
          con = pymysql.connect(host='127.0.0.1',port = 3306,user = 'root', password = 'root',
database = 'preowned',charset='utf8')
        with con:
            cur = con.cursor()
            cur.execute("select * FROM signup")
            rows = cur.fetchall()
            for row in rows:
                if row[0] == username and row[1] == password:
                    uname = username
                    option = 1
                    break
        if option == 1:
            context= {'data':'welcome '+username}
            return render(request, 'UserScreen.html', context)
        else:
            context= {'data':'Invalid login details'}
            return render(request, 'UserLogin.html', context)
```

CHAPTER – 6

SOFTWARE TESTING

6.1 Implementation and Testing

Implementation is one of the most important tasks in project is the phase in which one must be cautions because all the efforts undertaken during the project will be very interactive. Implementation is the most crucial stage in achieving successful system and giving the users confidence that the new system is workable and effective. Each program is tested individually at the time of development using the sample data and has verified that these programs link together in the way specified in the program specification. The computer system and its environment are tested to the satisfaction of the user.

Implementation

The implementation phase is less creative than system design. It is primarily concerned with user training, and file conversion. The system may be requiring extensive user training. The initial parameters of the system should be modifying because of a programming. A simple operating procedure is provided so that the user can understand the different functions clearly and quickly. The different reports can be obtained either on the inkjet or dot matrix printer, which is available at the disposal of the user. The proposed system is very easy to implement. In general implementation is used to mean the process of converting a new or revised system design into an operational one.

Testing

Testing is the process where the test data is prepared and is used for testing the modules individually and later the validation given for the fields. Then the system testing takes place which makes sure that all components of the system property function as a unit. The test data should be chosen such that it passed through all possible condition. Testing is the state of implementation which aimed at ensuring that the system works accurately and efficiently before the actual operation commence. The following is the description of the testing strategies, which were carried out during the testing period.

6.2 Types of Testing

System Testing

Testing has become an integral part of any system or project especially in the field of information technology. The importance of testing is a method of justifying, if one is ready to move further, be it to be check if one is capable to with stand the rigors of a particular situation cannot be underplayed and that is why testing before development is so critical. When the software is developed before it is given to user to use the software must be tested whether it is solving the purpose for which it is developed. This testing involves various types through which one can ensure the software is reliable. The program was tested logically and pattern of execution of the program for a set of data are repeated. Thus, the code was exhaustively checked for all possible correct data and the outcomes were also checked.

Module Testing

To locate errors, each module is tested individually. This enables us to detect error and correct it without affecting any other modules. Whenever the program is not satisfying the required function, it must be corrected to get the required result. Thus all the modules are individually tested from bottom up starting with the smallest and lowest modules and proceeding to the next level. Each module in the system is tested separately. For example the job classification module is tested separately. This module is tested with different job and its approximate execution time, and the result of the test is compared with the results that are prepared manually. The comparison shows that the results proposed system works efficiently than the existing system. Each module in the system is tested separately. In this system the resource classification and job scheduling modules are tested separately, and their corresponding results are obtained which reduces the process waiting time.

Integration Testing

After the module testing, the integration testing is applied. When linking the modules there may be chance for errors to occur, these errors are corrected by using this testing. In this system all modules are connected and tested. The testing results are very correct. Thus, the mapping of jobs with resources is done correctly by the system.

Acceptance Testing

When that user fined no major problems with its accuracy, the system passers through a final acceptance test. This test confirms that the system needs original goals, objectives and requirements established during analysis without actual execution which elimination wastage.

Test Case Id	Test Case Name	Test Case Desc.	Test Steps			Test Case Status	Test Priority
			Step	Expected	Actual		
01	Admin	Verify Admin is login or not	If Admin is not login	we cannot do any further operations	we can do further operations	High	High
02	User (Registration &User Login)	Verify User login or not	If User is not login	we cannot do any further operations	we can do further operations	High	High
03	Seller Module	Verify Seller Module is working or not	If Seller Module is not working	we cannot do any further operations	we can do further operations	High	High
04	Buyer Module	Verify Buyer Module is working or not	If Buyer Module is not working	We cannot run operation	We can Run the Operation	High	High
05	Feedback & Rating	Verify Feedback & Rating is working or not	If Feedback & Rating is not working	We cannot run operation	We can Run the Operation	High	High

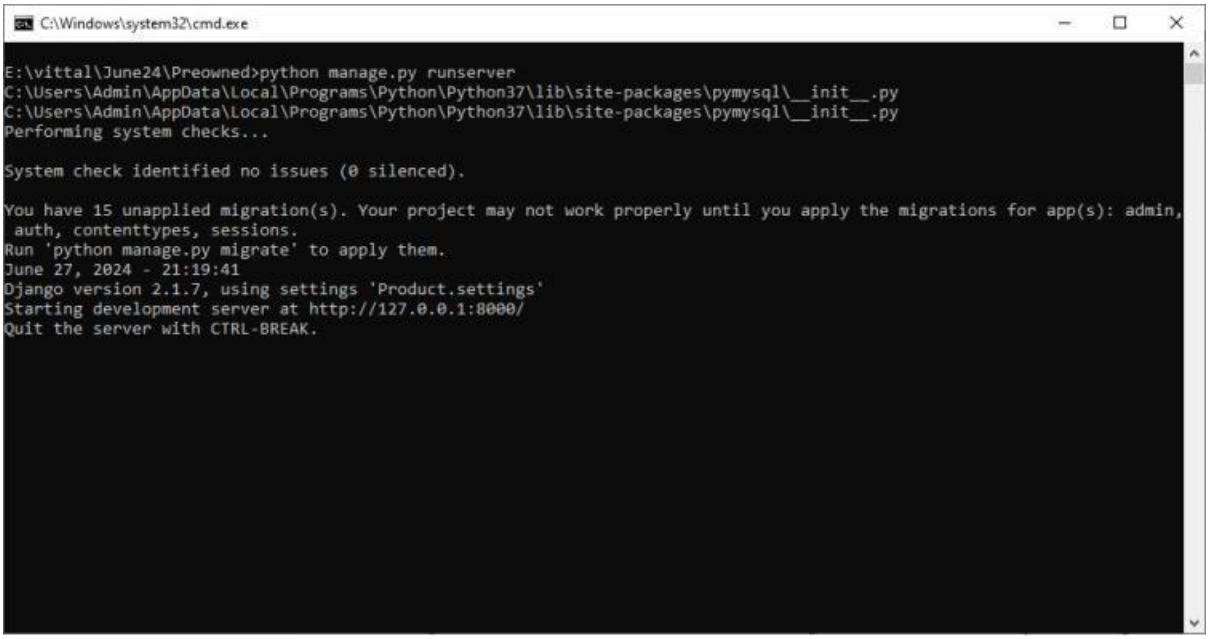
CHAPTER – 7

RESULTS AND OUTPUT SCREENS

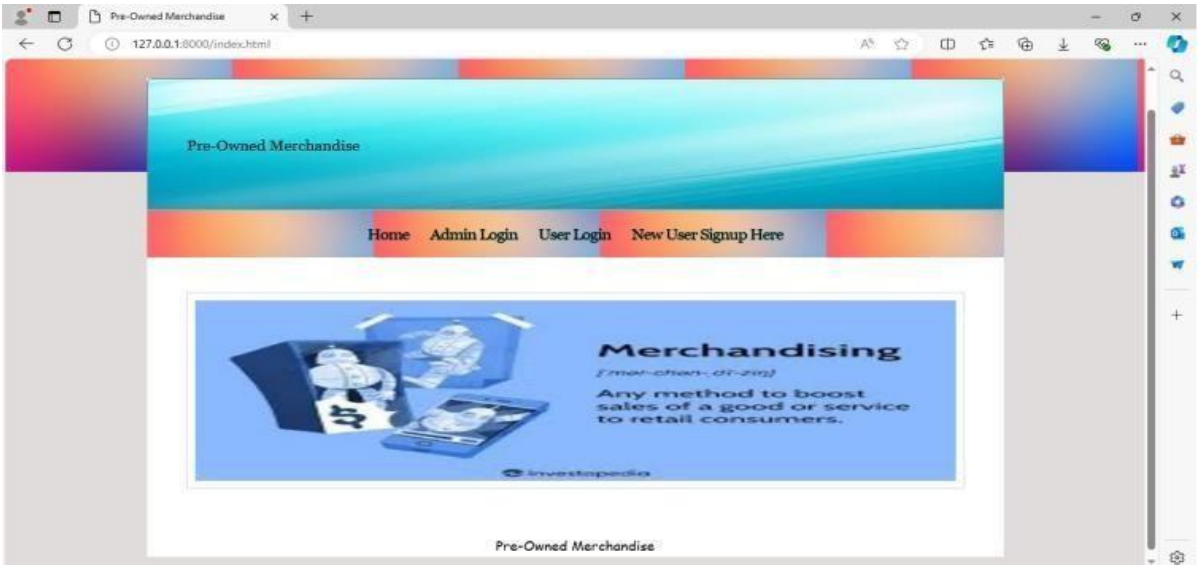
7.1 Output Screens

To run project, install python and then install all packages given in requirements.txt file and then install MYSQL database and then open MYSQL console and then copy content from DB.txt file and paste in MYSQL console to create database.

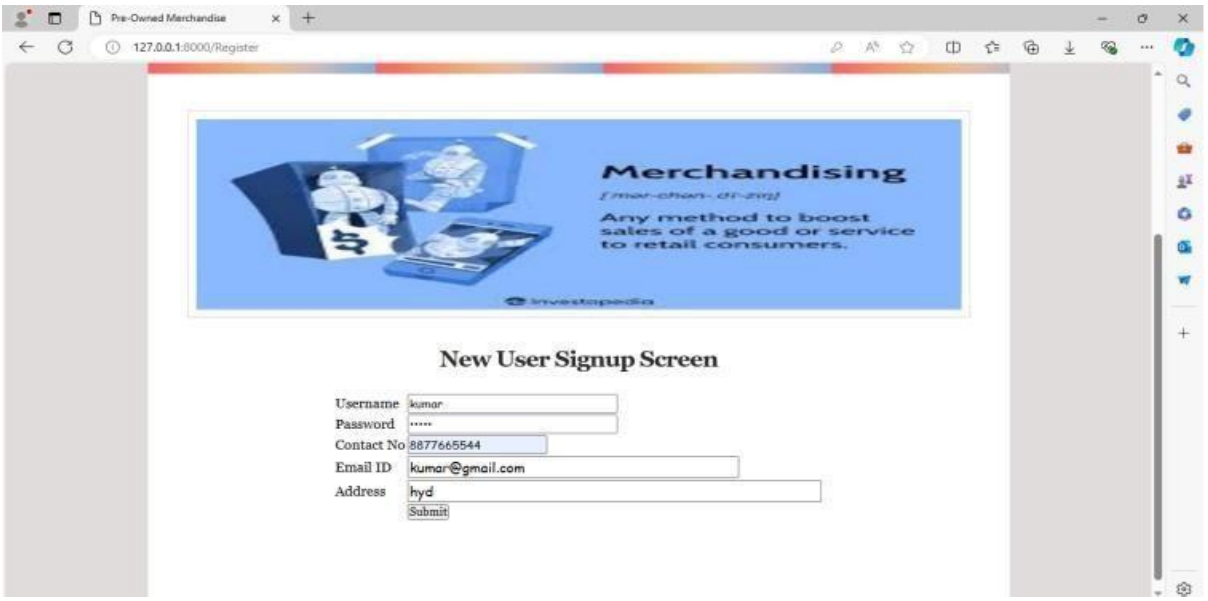
Now double click on 'run.bat' file to start python server and get below page.



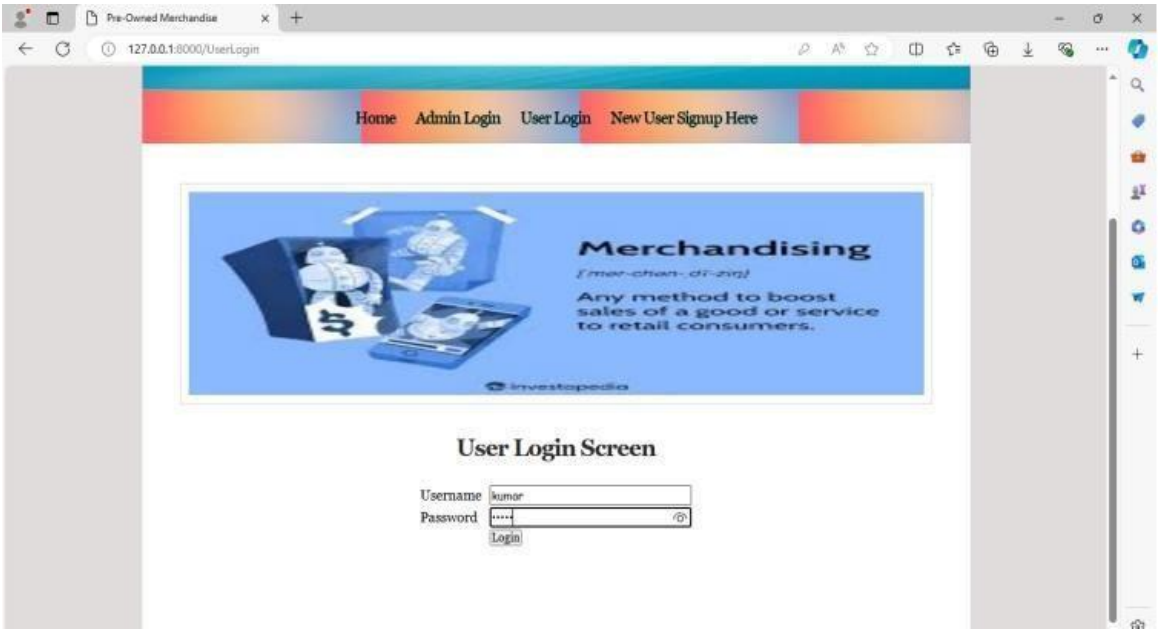
In above screen python server started and now open browser and enter URL as http://127.0.0.1:8000/index.html and press enter key to get below page



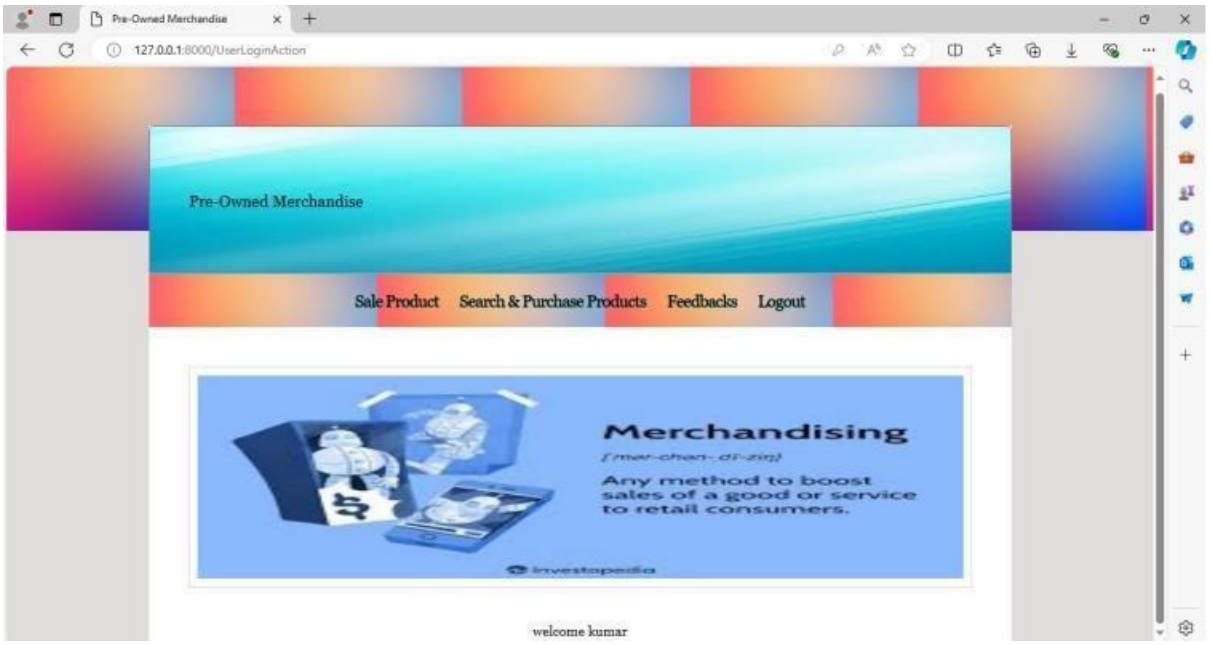
In above screen click on 'New User Signup Here' link to get below sign up page



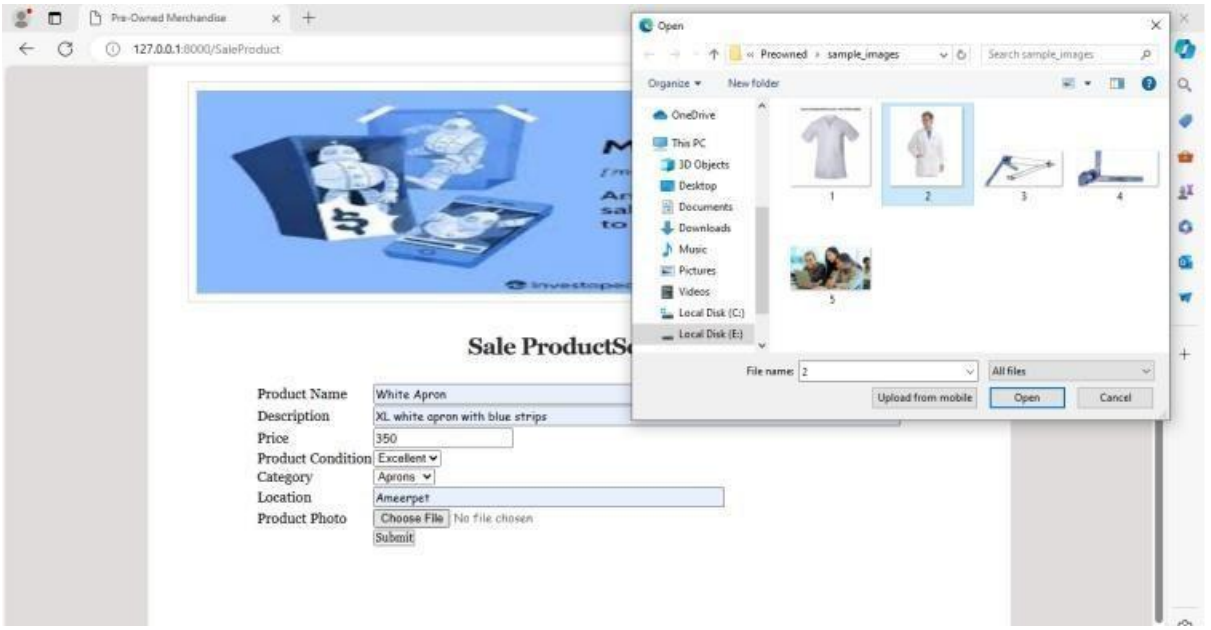
In above screen user is entering sign up details and then press button to get below page



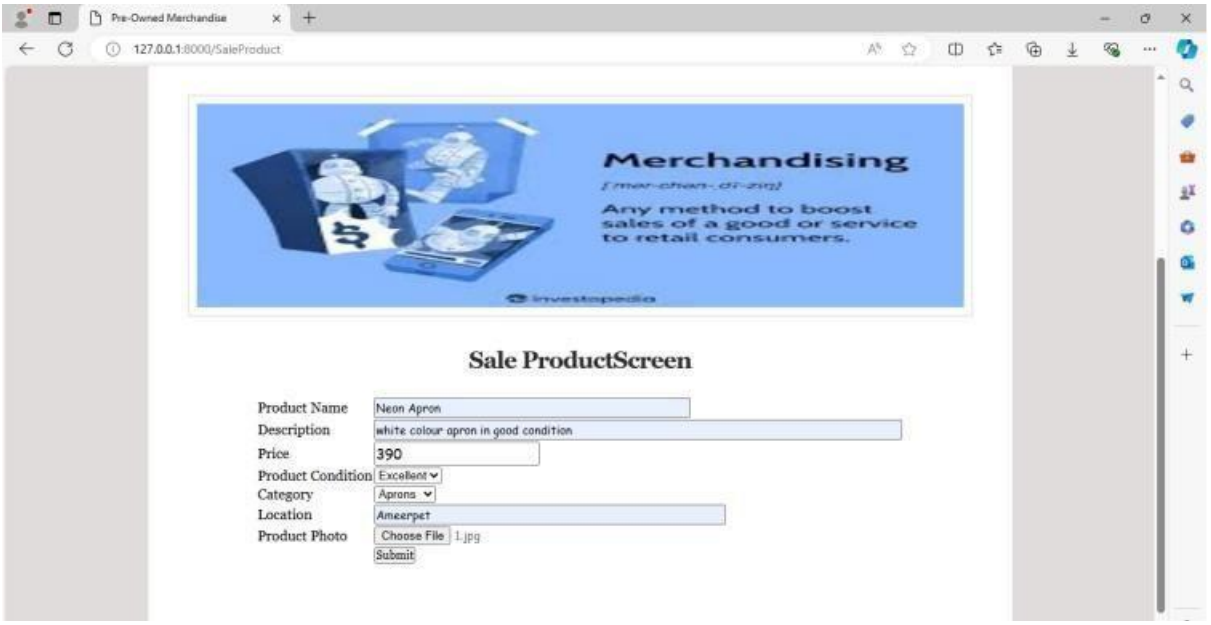
In above screen user is login and after login will get below page



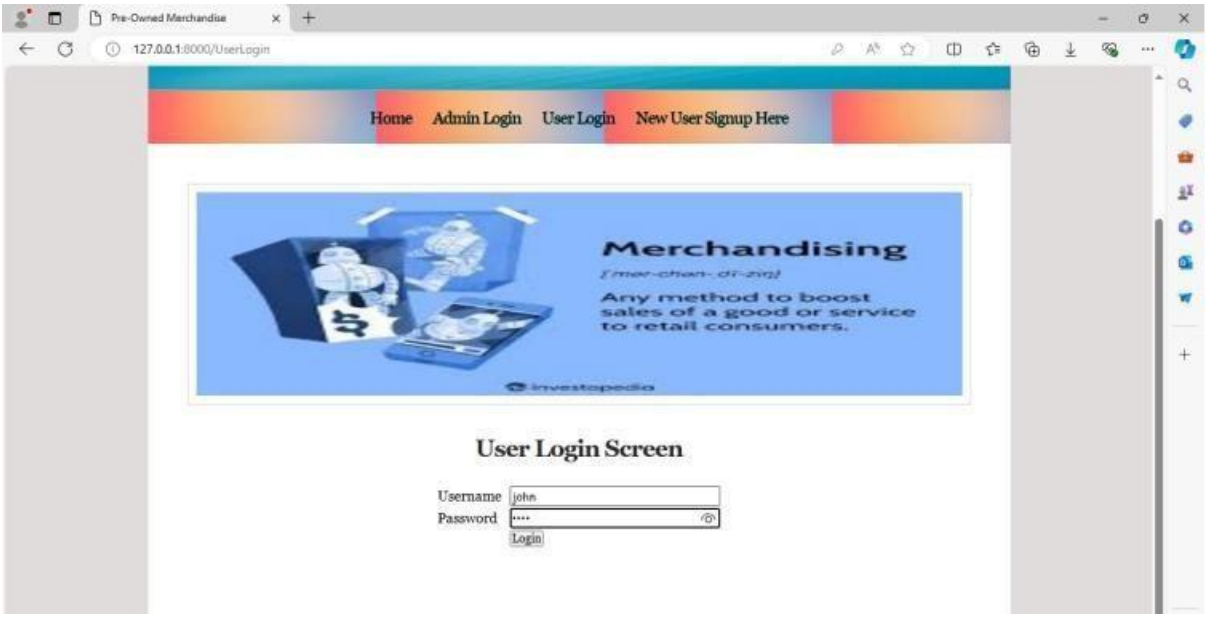
In above screen user can click on ‘Sale Product’ link to add product details like below screen.



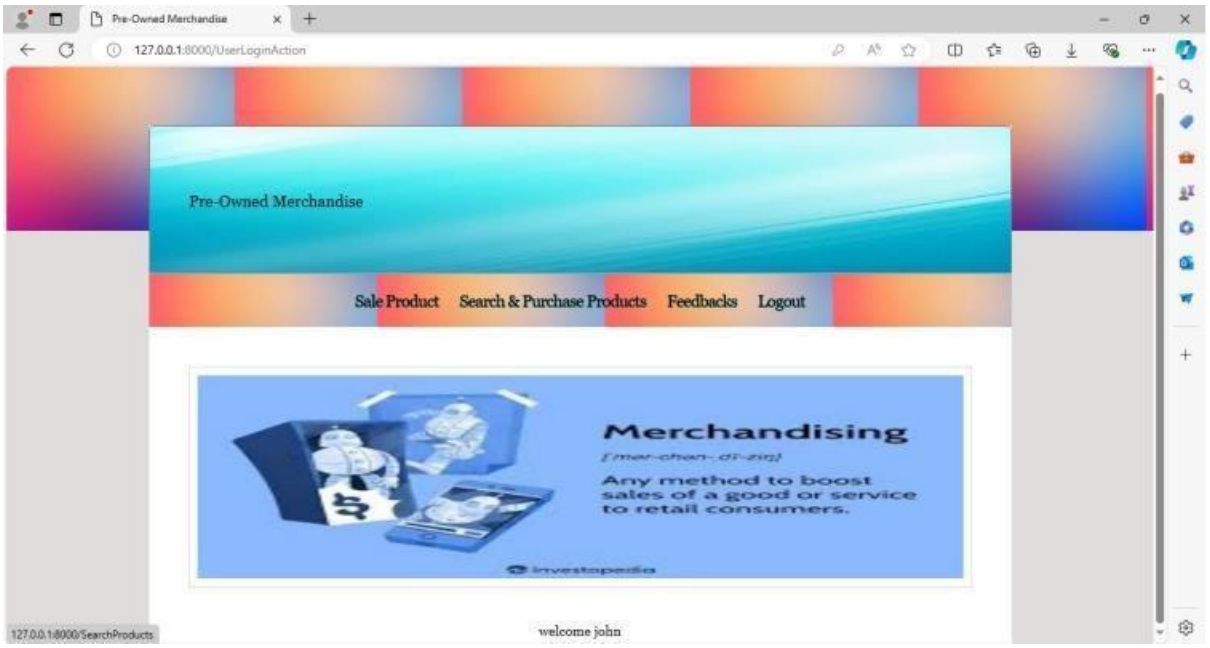
In above screen user is adding new product details and then uploading related images and now click on ‘Submit’ button to get below page



In above screen adding another product details and now logout and login as another user to view or search products



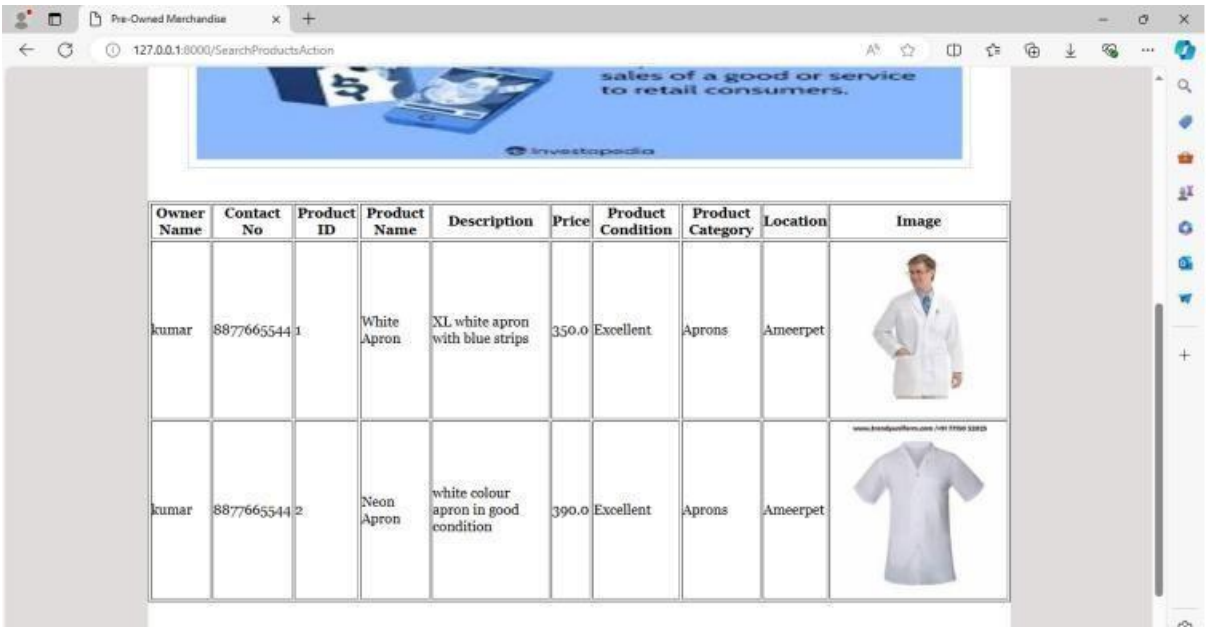
In above screen another user is login and after login will get below page



In above screen user can click on ‘Search & Purchase Products’ link to get below page



In above screen user is selecting desired product search criteria and then press button to get below search page

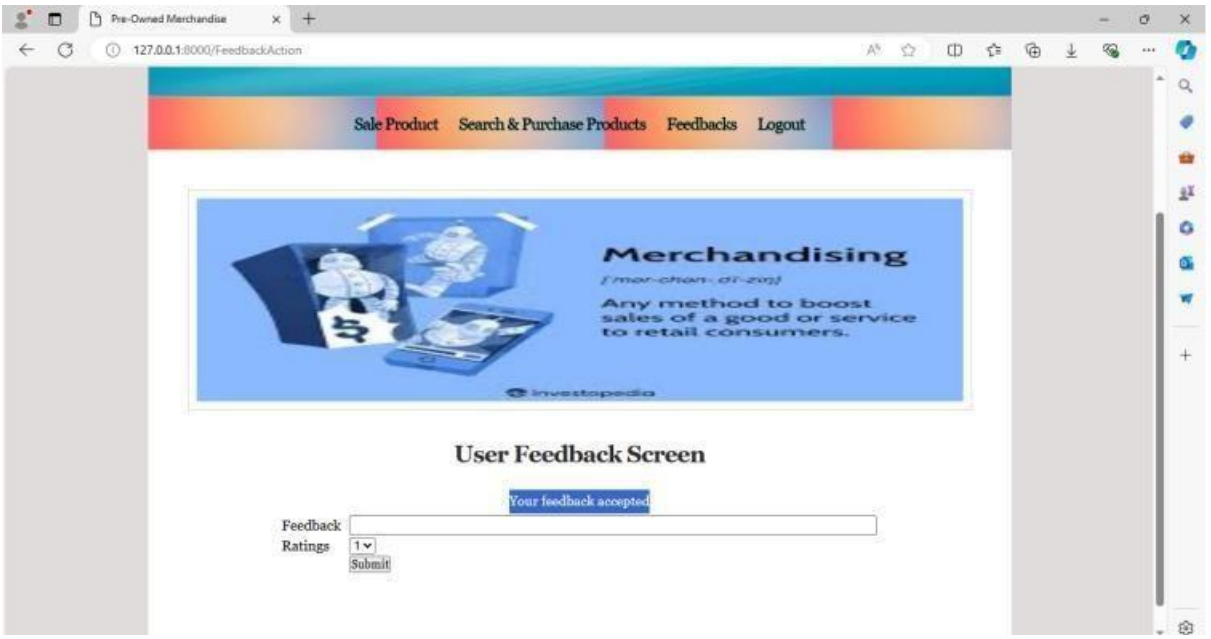


Owner Name	Contact No	Product ID	Product Name	Description	Price	Product Condition	Product Category	Location	Image
kumar	8877665544	1	White Apron	XL white apron with blue strips	350.0	Excellent	Aprons	Ameerpet	
kumar	8877665544	2	Neon Apron	white colour apron in good condition	390.0	Excellent	Aprons	Ameerpet	

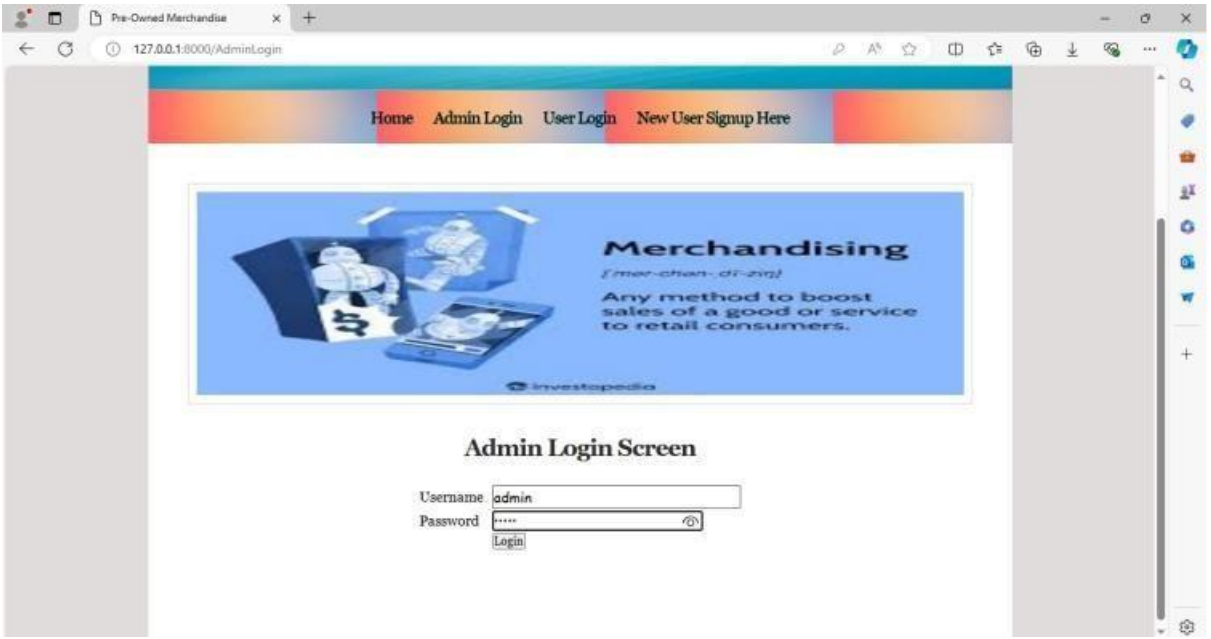
In above screen user can see all product details based on search criteria and user can see owner name and contact number to confirm purchase and now click on 'Feedback' link to give feedbacks.



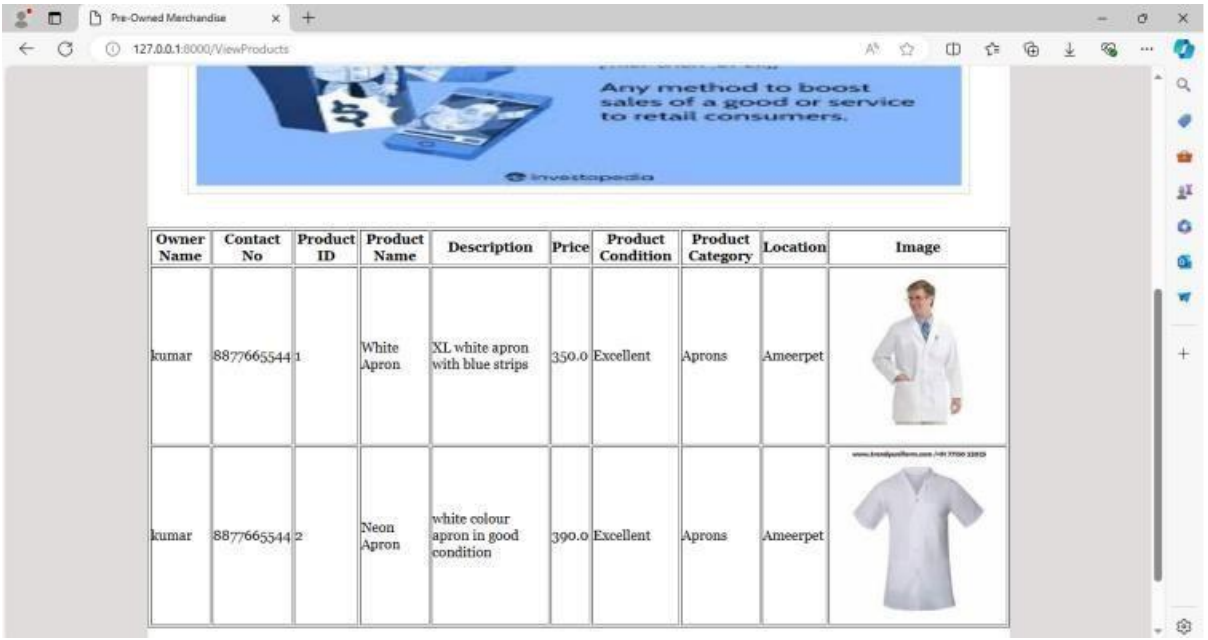
In above screen user will give ratings and feedback for selected product and then click on 'Submit' button to save feedback



In above screen user feedback accepted and now logout and login as ‘admin’ to view all details.



In above screen admin is login and after login will get below page



Owner Name	Contact No	Product ID	Product Name	Description	Price	Product Condition	Product Category	Location	Image
kumar	8877665544	1	White Apron	XL white apron with blue strips	350.0	Excellent	Aprons	Ameerpet	
kumar	8877665544	2	Neon Apron	white colour apron in good condition	390.0	Excellent	Aprons	Ameerpet	

In above screen admin can view list of available products and now click on 'View Feedback' link to view all feedbacks like below screen.



Username	Product ID	Feedback	Ratings
john	1	product condition was nice	5

In above screen admin can view all feedback from all users.

Similarly, by following above screens users can sale or purchase merchandise products

CHAPTER – 8

APPLICATIONS AND FUTURE ENHANCEMENTS

8.1 Applications

1. Helps users to buy and sell second-hand products easily through an online platform.
2. Useful for students and individuals to purchase affordable items like books, electronics, and furniture.
3. Can be used in college communities for exchanging or reselling used items.
4. Supports small-scale sellers to list and sell their products without needing a physical store.
5. Promotes reuse and recycling of goods, reducing environmental waste.
6. Enables local marketplace creation, allowing users to trade within nearby areas.
7. Helps users to find rare or discontinued products that are not easily available in new condition.
8. Provides a platform for quick disposal of unused items, saving storage space.
9. Encourages cost-effective shopping, especially for budget-conscious users.
10. Can be extended for business or startup models focusing on second-hand goods.
11. Facilitates peer-to-peer transactions without the need for intermediaries.
12. Useful for reselling seasonal or short-term use items like project materials or event supplies.
13. Helps in digital record keeping of products and user interactions for better tracking.
14. Can be adapted for specific categories like books exchange, gadgets resale, or furniture trading.
15. Supports community-based sharing economy, encouraging efficient use of resources.

8.2 Future Enhancements

1. Integration of Online Payment Gateway:
   Implement secure online payment options such as credit/debit cards, UPI, or net banking to enable real-time transactions within the platform.
2. Chat System Between Buyer and Seller:
   Add an inbuilt messaging feature to allow direct communication without sharing personal contact details.
3. Mobile Application Development:
   Develop an Android and iOS mobile app to increase accessibility and improve user convenience.
4. Product Recommendation System:
   Implement an AI-based recommendation engine to suggest products based on user preferences and browsing history.
5. Enhanced Security & Verification:
   Introduce OTP verification, email authentication, and user profile verification to improve platform security and build greater trust among users.
6. Advanced Search & Filters:
   Enhance the search functionality with filters such as price range, category, condition, and date of posting to improve user experience.
7. Wishlist Feature: Allow users to save products to a wish-list for future reference, helping them track items they are interested in.
8. Rating & Review System Improvement:
   Introduce detailed rating mechanisms with multiple criteria (product quality, seller behavior, etc.) to increase transparency.

CHAPTER – 9

CONCLUSION

The Pre-Owned Merchandise system is successfully designed and developed as a web-based platform that simplifies the process of buying and selling second-hand products. The project provides an organized and efficient environment where users can easily interact, list products, and explore available items without the complexities of traditional methods.

The system incorporates essential functionalities such as user authentication, product management, search mechanisms, and feedback features, ensuring a smooth and reliable user experience. The inclusion of an admin module further strengthens the system by enabling effective monitoring and control over users and product data. The structured database design ensures proper storage, retrieval, and management of information.

This project also demonstrates the practical implementation of key concepts such as web application development, database connectivity, and data validation. It highlights how technology can be used to create a transparent and accessible marketplace for second-hand goods. While certain advanced features like online payments and real-time communication are not included, the current system provides a strong foundation for future enhancements.

Overall, the project achieves its objective of delivering a simple, secure, and user-friendly platform for pre-owned merchandise transactions, promoting efficient resource utilization and supporting a sustainable approach to buying and selling.

REFERENCES:

1. H. T. Cheng, "Wide deep learning for recommender systems," in *Proceedings of the 1st workshop on deep learning for recommender systems*, pp. 7–10, 2016.
2. S. Pudaruth, "Predicting the price of used cars using machine learning techniques," *Int. J. Inf. Computer. Techno*, pp. 753–764, 2014.
3. I. Raykhel, and D. Ventura, "Real-time automatic price prediction for ebay online trading," in *Twenty-First IAAI Conference*, 2009.
4. Q. You, R. Pang, L. Cao, and J. Luo, "Image-based appraisal of real estate properties," *IEEE Transactions on Multimedia*, pp. 2751–2759, 2017.
5. S. Peterson, and A. Flanagan, "Neural network hedonic pricing models in mass real estate appraisal," *Journal of real estate research*, pp. 147–164, 2009.
6. S. Law, B. Paige, and C. Russell, "Take a look around: using street view and satellite images to estimate house prices," *ACM Transactions on Intelligent Systems and Technology (TIST)*, pp. 1–19, 2018.
7. R. Schlosser, and M. Boissier, "Dynamic pricing under competition on online marketplaces: A data-driven approach," in *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery Data Mining*, pp. 705–714, 2018.
8. V. Chiarazzo, L. Caggiani, M. Marinelli, and M. Ottomanelli, "A Neural Network based model for real estate price estimation considering environmental quality of property location," *Transportation Research Procedia*, pp. 810–817, 2014.
9. P. Ye, "Customized regression model for airbnb dynamic pricing," in *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery Data Mining*, pp. 932–940, 2018.
10. Y. Zhang, "Visual search at alibaba," in *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery Data Mining*, pp. 993–1001, 2018.
11. X. Ji, W. Wang, M. Zhang, and Y. Yang, "Cross-domain image retrieval with attention modeling," In *Proceedings of the 25th ACM international conference on Multimedia*, pp. 1654–1662, 2017.

12. L. Xie, J. Shen, and L. Zhu, "Online cross-modal hashing for web image retrieval," in *Thirtieth AAAI Conference on Artificial Intelligence*, pp. 294–300, 2016.
13. X. Zhang, H. Lai, and J. Feng, "Attention-aware deep adversarial hashing for cross-modal retrieval," in *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 591–606, 2018.
14. P. Xu, "Cross-modal subspace learning for fine-grained sketch-based image retrieval," *Neurocomputing*, pp. 75–86, 2018.
15. T. Yu, J. Yuan, C. Fang, and H. Jin, "Product quantization network for fast image retrieval," in *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 186–201, 2018.
16. Y. Qi, Y. Z. Song, H. Zhang, and J. Liu, "Sketch-based image retrieval via siamese convolutional neural network," in *2016 IEEE International Conference on Image Processing (ICIP)*, pp. 2460–2464, 2016.